XIMOD Architect — XML Builder

User Manual

The XML installer, designed for your Mods.

Skyrim · Fallout · Starfield · Palworld · and many more

Version 2.2.2

Cross-platform: Windows · Linux · macOS



Contents

1. Overview 5

1.1 What is XIMOD Architect? 5

1.2 The FOMOD format 5

1.3 Installation and requirements 6

Windows 6

Linux 6

macOS 6

1.4 File structure 6

2. Getting started 8

2.1 Launching the application 8

2.2 Interface overview 8

2.3 Creating a new project 9

2.4 Opening and saving a project 9

2.5 First launch 9

2.6 Free windows and keyboard navigation 9

3. The menus 11

3.1 File menu 11

3.2 Edit menu 12

3.3 Project menu 12

3.4 Tools menu 13

3.5 Help menu 14

4. Features in detail 15

4.1 The Mod Info tab 15

4.2 The Install Steps tab 15

Groups and selection types 16

Options and default types 16

Plugin files 16

Group / page destination 17

Condition flags 17

4.3 The Required Installs tab 17

4.4 The Conditional Installs tab 18

4.5 The "Settings" window 18

4.6 Pre- and post-save scripts 19

Example — pre-save script (Windows) 20

Example — post-save script (Windows) 20

Linux / macOS equivalent 20

4.7 The "About" window 20

4.8 Generating the FOMOD files 20
4.9 Translation editor 21
4.10 XML editor 23
4.11 Font management 23
4.12 Merging two FOMODs 25
4.13 Exporting the distribution archive 25
4.14 The "Properties" window 25
4.15 Command-line mode (CLI) 27
4.16 Checking for updates 27
5. FOMOD tag reference 28
5.1 The info.xml file 28
5.2 The ModuleConfig.xml file 28
Root and header 28
Files and folders 29
Steps and groups 29
Options (plugins) 29
Dependencies and conditions 30
6. The ModuleConfig.xml file explained in detail 31
6.1 How the installer reads the file 31
6.2 The root and the header 31
6.3 Installing files 31
6.4 Steps, groups and options 32
6.5 An option's type and its behaviour 33
6.6 Flags, dependencies and conditional installs 33
7. ModuleConfig.xml examples 35
7.1 Minimal example — required files 35
7.2 Exclusive choice — SelectExactlyOne 35
7.3 Conditional install — flags 35
7.4 One page, several steps — SelectAny + flags 36
7.5 Preset with customisable defaults 38
7.6 Introduction and closing pages 41
8. Version 2 — new features 43
8.1 Destination-conflict detection 43
8.2 7z export 43
8.3 Reusable templates 43
8.4 New from folder 43
8.5 Drag-and-drop file assignment 43
8.6 FOMOD comparison 43
8.7 Visual condition editor 43

8.8 Image checks and optimization ... 44
8.9 Command-line additions ... 44
8.10 Background export and validation ... 44
8.11 Crash recovery ... 44
8.12 A clearer interface ... 44
8.13 Translating an existing FOMOD ... 44
8.14 Opening a mod archive ... 45
8.15 Import fidelity ... 46
8.16 Backups and recovery ... 46
8.17 Masters as automatic conditions ... 46
8.18 Light plugins (ESL) and the plugin report ... 47
8.19 Richer simulator and scenarios ... 47
8.20 Install size per option ... 47
8.21 Nexus description generator ... 47
8.22 An editable condition editor ... 48
8.23 Project strings table ... 48
8.24 Reading BSA and BA2 archives ... 48
8.25 Nested condition groups and version conditions ... 49
8.26 Editing the games and their categories ... 49
8.27 Tree colours ... 49
8.28 File dialogs remember where you were ... 50
8.29 Preset wizard ... 50
8.30 Cut, copy and paste between FOMODs ... 51
8.31 Renaming a source on disk ... 51
8.32 Finding missing files ... 51
Appendices ... 52
Appendix A — The Config.ini file ... 52
Appendix B — Keyboard shortcuts ... 52
Appendix C — Glossary ... 53
Appendix D — Complete FOMOD example ... 55
D.1 info.xml ... 55
D.2 ModuleConfig.xml ... 55
Appendix E — License and credits ... 59

1. Overview

1.1 What is XIMOD Architect?

XIMOD Architect is a cross-platform tool for creating FOMOD installers for game mods — Skyrim, Fallout, Starfield and any other game that Vortex or Mod Organizer 2 can manage. It lets you visually build a complete installation wizard — steps, option groups, options, conditional files — and then automatically generate the standardized XML files that mod managers know how to interpret.

FOMOD is a mod-manager format, not a game format: the installers you build work with every Bethesda game and with most other games that Vortex or Mod Organizer 2 can manage — if a game can be modded with Vortex, it can use a FOMOD installer. The built-in list in the Categories.json file (eleven Bethesda titles: the three editions of Skyrim — Legendary, Special Edition and VR — Fallout 3, Fallout: New Vegas, Fallout 4 and Fallout 4 VR, Oblivion and Oblivion Remastered, Morrowind and Starfield) only fills in the game and the Nexus categories of info.xml; it does not limit where an installer can be used. The list is external and editable: Tools → Games / categories… (§ 8.26) adds any other game without recompiling the application.

The software is a rewrite in the Rust language of the original FOMOD Creation Tool. This overhaul brings better performance, identical behaviour on Windows, Linux and macOS, and a modern interface. XIMOD Architect also stands out for its very advanced internationalization system: the interface can be translated into a large number of languages, with dynamic loading of the fonts suited to each writing system.

The main features are as follows:

- creation of complete FOMOD installation packages;
- multi-step installation wizards;
- option groups with different selection types (exactly one, at least one, any, and so on);
- conditional file installation based on the user's choices;
- dependency patterns that drive an option's type;
- a system of "flags" (condition flags) controlling the flow of the installation;
- custom scripts run before or after saving;
- a multilingual interface and dynamic fonts;
- a native, transparent splash screen.

1.2 The FOMOD format

FOMOD ("Fallout Mod") is the standard format for mod installers for Bethesda games, and mod managers such as Vortex and Mod Organizer 2 use it for most other games as well. A FOMOD installer is based on two XML files gathered in a subfolder named fomod:

- `info.xml` — the mod metadata: name, author, version, website, description, category.
- `ModuleConfig.xml` — the installation configuration: steps, groups, options, files, conditions.

The files produced by XIMOD Architect are compatible with the main mod managers:

- Mod Organizer 2 (MO2);
- Vortex;
- Nexus Mod Manager (NMM).

> **Note —** *The name of the fomod folder as well as the names info.xml and ModuleConfig.xml are imposed by the specification and recognized as-is by mod managers. They must never be renamed.*

1.3 Installation and requirements

System requirements (indicative):

Item	Recommendation
System	Windows 10 or later, Linux (recent glibc) or macOS 11 or later.
Display	Graphics card compatible with OpenGL 3.3 (egui's "glow" renderer).
RAM	A few hundred MB — the application is lightweight.
Disk space	A few tens of MB, depending on the Noto fonts and the flags provided in assets/.

Windows

Place in the same folder the executable ximod-architect.exe, the assets folder (data, locales, fonts, images, icons) and, optionally, the file assets/images/splash.png for the splash screen. The application icon is embedded in the executable at compile time. Then run ximod-architect.exe.

Linux

An installation script is provided in packaging/linux:

```
# System-wide installation (requires sudo)
sudo ./packaging/linux/install.sh

# Installation for the current user
./packaging/linux/install.sh --user
```

Building from source requires Rust 1.99.0 or later and, on Linux, the following development libraries: libgtk-3-dev, libxcb-render0-dev, libxcb-shape0-dev, libxcb-xfixes0-dev, libspeechd-dev, libxkbcommon-dev, libssl-dev and libx11-dev.

macOS

A script creates an application bundle (.app) ready to distribute:

```
./packaging/macos/create-bundle.sh
```

The "XIMOD Architect.app" bundle is generated in the dist folder. A disk image (.dmg) can then be produced with the hdiutil utility.

1.4 File structure

The file tree accompanying the executable is as follows:

```
ximod-architect(.exe)            → executable
Config.ini                       → configuration (created on first launch)
assets/
   data/
      Categories.json            → games and Nexus categories
      Languages.json             → 6,777 languages (endonym, ISO codes, font)
      Countries.json             → 244 countries (names, flag, official languages)
      CountryLanguages.json      → languages spoken by country
   locales/                      → translations (one folder per language)
      eng/main.ftl
      fra/main.ftl
      …                          → 32 languages provided
   fonts/                        → Noto fonts (one per writing system)
   icons/                        → application icons
   images/
      splash.png                 → splash screen
      svg/                       → country flags (244 SVG files)
```

2. Getting started

2.1 Launching the application

At startup, XIMOD Architect first displays a native, transparent splash screen, with a fade effect, whose duration is adjustable (two seconds by default; the screen can be disabled). The main window then opens, centred on the primary display. Its initial size comes from the configuration file (1280 × 800 by default) and cannot go below 800 × 600. The title bar shows the application name followed by its version number.

2.2 Interface overview

The main window is made up of five areas:

- **the menu bar at the top — File, Edit, Project, Tools and Help — and, just below it, the toolbar with the everyday actions (New, Open, Save, Undo, Redo, Validate, Preview, Export); see chapter 3;**
- **the project tree on the left: Mod information, then the installation steps with their groups and options, the required files and the conditional installs. Click a node to edit it, right-click it for its actions (add, duplicate, move, delete, save as template), and drag an option onto another group, a group onto another step, or a step before another step to move it; the + buttons next to "Installation steps" and "Conditional installs" add an element directly;**
- **the inspector in the centre, which shows the editor of the selected node: the mod information, a step (name, visibility conditions, page destination, list of its groups), a group (name, selection type, group destination, list of its options), an option (description, image, type, flags, dependency patterns, files), the required files or a conditional set; a breadcrumb at the top of the inspector lets you go back up one level;**
- **the Problems panel at the bottom, shown after Project → Validate FOMOD: every finding is listed with its severity, and clicking it selects the node concerned in the tree; the tree itself shows a small error/warning counter next to the nodes concerned;**
- **the status bar at the very bottom: on the left, the latest message (at startup: "Ready"); on the right, the number of steps and options and, when a project has unsaved changes, a dot followed by "Modified". Successes, warnings and errors also appear as notifications in the top-right corner.**

The appearance of the interface (dark, light or system theme, font size, language) is set in the Settings window described in § 4.5.

In the tree, a click selects a node and shows it in the inspector; a double-click on a step or a group does the same and opens it. The right-click menu of a step offers Expand (All, Selected, from Selected — the step and everything under it) and Collapse (same three choices), and the menu of a group offers Expand and Collapse (All, Selected), ahead of the usual add / duplicate / move / delete commands. Each tab keeps its own Problems panel: the findings shown always belong to the FOMOD loaded in the active tab, and a validation started in one tab finishes in that tab even if you switch to another meanwhile.

The tree can also be driven from the keyboard while no text field has the focus: ↑ and ↓ move a cursor ring from row to row (it starts on the selected node), Enter selects the row under it, → unfolds a closed step or group (and, on an open one, moves to its first child), ← folds an open one (and, on a closed row or an option, goes back to its parent). The ring is scrolled into view as it moves.

2.3 Creating a new project

Creating an installer generally follows the steps below:

- **New project** — File menu → New, then choose the root directory containing your mod's files ("Browse…" button in the Mod Info tab).
- **Fill in the information** — in the Mod Info tab, enter the mod name, author, version, game, category, website, header image and description.
- **Build the wizard — in the Install Steps tab, add steps, then option groups, then options.**
- **Associate the files — for each option, add the files or folders to install, then set their destination in the table if needed (it is empty by default, which installs to the root of the game's Data folder).**
- **Define the conditions** — use flags and conditional installs to adapt the installation to the user's choices.
- **Save** — File menu → Save (Ctrl+S): XIMOD Architect creates the fomod folder and writes info.xml and ModuleConfig.xml into it.

2.4 Opening and saving a project

Saving always generates the fomod subfolder under the root directory, then writes the two XML files into it in UTF-8. The Save command stays disabled as long as no root directory has been defined.

Two opening modes are offered. "Open folder…" asks for the root directory and loads the project from its fomod subfolder. "Open file…" asks directly for an XML file: the root directory is then inferred (the file is assumed to be located in a fomod subfolder). When reading, XIMOD Architect automatically recognizes the file encoding (UTF-8, with or without a byte order mark, as well as UTF-16), which makes it possible to reopen installers produced by other tools.

Each project opened successfully is added to the top of the recent files list, accessible through File → Recent and manageable in the Recent Files tab of the Settings.

2.5 First launch

On the very first launch, XIMOD Architect starts in English and automatically opens the Settings window, so that the user can choose their country and then their language.

This behaviour is driven by the `FirstStart` key in the Config.ini file: it is `0` as long as the initial configuration has not been saved, and switches to `1` as soon as "Save" is clicked for the first time. Subsequent launches then start directly in the chosen language.

If the user closes the window with "Cancel", the key stays at 0 and the window reopens at the next launch.

2.6 Free windows and keyboard navigation

Several tools open in **free (independent) windows**: the "Country selection" picker, the "Properties" window, the translation editor and the XML editor. Each one is freely movable and resizable, including on a second screen, and works alongside the main window.

Centring and persistence — the first time it is opened, a free window is centred on the XIMOD main window. If the user moves or resizes it, its position and size are remembered when it is closed (and when the application exits), then restored the next time it opens. These values are kept in Config.ini,

under the [WindowPositions] section (see Appendix A). The main window itself remembers its size but not its position: it stays centred at every launch.

Closing with Esc — the Esc key closes the active free window as well as the main window's modal dialogs (Settings, About, confirmation, validation report, scripts); for a confirmation, Esc acts as "Cancel" and never runs the action. Two exceptions: the translation editor (to avoid closing it accidentally while typing) and the XIMOD main window do not close on Esc.

Keyboard navigation — the lists in the "Properties" window, the translation editor table and the flag grid of the country picker can be operated entirely from the keyboard: arrow keys (and the numeric keypad), Page Up/Page Down, Home/End, Enter and Tab/Shift+Tab. The details, specific to each window, are described in the relevant sections and summarised in Appendix B.

Clearing filters — every "Filter:" field has, to its right, a clear icon (red cross) that empties the filter with a single click.

3. The menus

The menu bar contains five menus: File, Edit, Project, Tools and Help. Below it, a toolbar gives one-click access to the everyday actions (New, Open, Save, Validate, Preview, Export). The shortcuts shown are also active globally (the Ctrl key corresponds to Cmd on macOS); they are only ignored while a modal dialog is open. Modal dialogs dim the rest of the window; Enter triggers their primary button and Esc cancels.

3.1 File menu

Command	Shortcut	Description
New	Ctrl+N	Opens a new blank project in a new tab (the current tab is replaced only if it is still pristine).
New from folder…	—	Scans a mod folder and proposes a skeleton where each subfolder becomes one option. See § 8.4.
Open folder…	Ctrl+O	Chooses a root directory and loads the project from its fomod subfolder.
Open file…	Ctrl+Shift+O	Chooses an info.xml or ModuleConfig.xml directly; the root directory is inferred. A file that is not inside a fomod folder is refused.
Open archive…	—	Extracts a .zip or .7z mod archive (as downloaded from Nexus) into the working folder of the configuration directory, then opens its FOMOD. An archive already extracted is reused. .rar archives are refused: extract them with 7-Zip first. Dropping an archive on the window does the same. See § 8.14.
Save	Ctrl+S	Generates fomod/info.xml and fomod/ModuleConfig.xml. If the project has no root directory yet, a folder is asked for first.
Save as…	Ctrl+Shift+S	Chooses a (new) root directory for the project, then saves it there.
Restore a backup…	—	Lists the previous versions of the FOMOD XML files kept under fomod/backups, with the number of differences from the current project; each can be compared or restored into the editor. Disabled when the project has no backup yet. See § 8.16.
Merge FOMOD…	—	Adds the steps, required files and conditional installs of another FOMOD to the current project. Disabled as long as no root directory is defined. See § 4.12.
Export distribution archive…	—	Writes the FOMOD then compresses the whole root directory into a .zip or .7z archive ready to publish, in the background with a progress window. See § 4.13 and § 8.10.
Close FOMOD	—	Closes the active tab, asking first if it has unsaved changes.
Close all FOMODs	—	Closes every tab, asking first if any has unsaved changes.

Command	Shortcut	Description
Recent	—	Submenu listing the most recently opened projects. Shows "(empty)" if the list is empty.
Quit	Ctrl+Q	Closes the application. Unsaved projects are listed and can be saved first; the configuration is saved on exit.

3.2 Edit menu

Command	Shortcut	Description
Undo	Ctrl+Z	Reverts the last change to the project (keystrokes in the same field are merged). Disabled when there is nothing to undo.
Redo	Ctrl+Y	Re-applies the last undone change (Ctrl+Shift+Z also works).
Cut	Ctrl+X	Removes the step, group or option selected in the tree and keeps it in the clipboard. Edit → Undo puts it back. See § 8.30.
Copy	Ctrl+C	Copies the selected step, group or option, with everything it contains, to the clipboard — and to the system clipboard. See § 8.30.
Paste	Ctrl+V	Inserts the clipboard next to the selection: a step after the selected step, a group into the selected step, an option into the selected group. See § 8.30.

3.3 Project menu

Everything that acts on the FOMOD being edited.

Command	Shortcut	Description
Validate FOMOD	—	Checks the project and the conformity of ModuleConfig.xml to the ModConfig 5.0 schema, verifies the referenced files, destination conflicts and images (in the background), then shows a detailed report. See § 4.8.
Preview installer…	—	Simulates the installer as a mod manager would show it. See § 4.11.
Find missing files…	—	Lists the files, folders and images the project uses that no longer exist and proposes where they went. See § 8.32.
Condition editor…	—	Every condition flag with where it is set and used (clickable), rename or remove a flag everywhere, and a condition builder for the selected node. See § 8.7 and § 8.22.
Translate a FOMOD…	—	Opens the window that translates an existing installer (the active project, a folder or an archive) without altering its XML structure. See § 8.13.

Command	Shortcut	Description
Compare with…	—	Compares the project against another FOMOD and lists what changed. See § 8.6.
Templates…	—	Saves the selected step as a reusable template, or inserts a saved template. See § 8.3.
Preset wizard…	—	Generates a "preset + customise" installer on the current project: an installation-mode page with the presets, feature pages shown on demand, dynamic defaults and conditional installs. See § 7.5 and § 8.29.
XML Editor	—	Submenu giving access to info.xml and ModuleConfig.xml. See § 4.10.

3.4 Tools menu

Settings and utilities that are not tied to a particular project.

Command	Shortcut	Description
Settings…	Ctrl+,	Opens the Settings window (country, language, theme, font, etc.). See § 4.5.
Tree colours…	—	Chooses one colour per kind of row of the project tree (steps, groups, options…), for each theme. See § 8.27.
Pre-Save Script…	—	Edits the script run before saving. See § 4.6.
Post-Save Script…	—	Edits the script run after saving. See § 4.6.
Plugin report…	—	Table of every plugin file (.esp, .esm, .esl) installed by the project: kind, light flag, masters (missing ones highlighted), new records against the light-plugin limit and ESL eligibility. See § 8.18.
Archive contents…	—	Lists the files packed in a Bethesda .bsa or .ba2 archive (format, sizes, compression) with a search field. See § 8.24.
Nexus description…	—	Generates the Nexus Mods page text (BBCode or Markdown) from the project: requirements, installation options, installation note and changelog against a previous version; copy or save it. See § 8.21.
Project strings…	—	Table of every name and description of the project, with search, find/replace and duplicate detection; texts are edited in place. See § 8.23.
Translate the interface…	—	Opens the interface translation editor. See § 4.9.
Properties…	—	Opens the read-only explorer of the country / language database. See § 4.14.
Games / categories…	—	Opens the games editor: add, correct or remove a game and its Nexus categories (name, slug, base masters, light-plugin limit) without editing Categories.json by hand. See § 8.26.

3.5 Help menu

Command	Shortcut	Description
User manual	F1	Opens this manual (PDF) in the language of the interface, from the Manuals folder shipped with the application.
Check for updates…	—	Queries the latest published version and offers the download links.
About	—	Displays the "About" window (name, version, license). See § 4.7.

4. Features in detail

4.1 The Mod Info tab

This tab brings together the mod metadata and the choice of root directory.

Workspace — the "Root directory:" line shows the working folder (or "(Select a directory)" if it is not defined). The "Browse…" button lets you choose it.

The metadata fields are then as follows:

Field	Role
Mod Name	Displayed name of the installer (required).
Author	Name of the mod author.
Version	Version number (1.0.0 by default for a new project).
Game Name	Target game, chosen from a list drawn from Categories.json.
Category	Category of the mod. Depends on the chosen game; otherwise a default list is offered (Animation, Armour, Audio, Textures, Weapons, etc.).
Website	Address of the mod's page.
Header Image	Image shown at the top of the installer. "Browse…" and "Clear" buttons; a preview is displayed. The path is stored relative to the root directory.
Description	Presentation text of the mod (multiline area).

> **Note —** *Any change to a field marks the project as "Modified" in the status bar.*

Nexus link — next to the "Game Name" selector, a "Nexus ↗" button opens, directly in the browser, the Nexus Mods page of the chosen game (built from its "slug"). It is active only if the selected game has a slug.

The "Game Name" and "Category" lists come from the external file assets/data/Categories.json, editable without recompilation. Each game is described there by an identifier, a name, a Nexus "slug" and its list of categories:

```
{
  "games": {
    "skyrimSpecialEdition": {
      "name": "The Elder Scrolls V: Skyrim Special Edition",
      "nexusSlug": "skyrimspecialedition",
      "categories": ["Animation", "Armour", "Audio", "…"]
    }
  }
}
```

4.2 The Install Steps tab

This is the heart of the installation wizard. It is based on a three-level hierarchy: Steps → Groups → Options, which is exactly the shape of the project tree on the left. Add a step with the + button next to “Installation steps” (named “Step N” by default) or from the empty inspector; select a step, a group or an option in the tree to edit it in the centre. The step editor shows the name (editable), the “Delete step” button (the only deletion that asks for confirmation: everything else can be undone with Ctrl+Z), the visibility conditions, the page destination and the list of groups with an “Add group” button; the

group editor shows the name, the selection type, the group destination and the list of options with an “Add option” button; the option editor is described below.

Groups and selection types

Each group has a name and a selection type that determines how many options the end user will be able to check. The five available types are:

Type	Meaning
Exactly one (required) — SelectExactlyOne	The user must choose exactly one option.
At most one — SelectAtMostOne	At most one option (or none).
Any number — SelectAny	Any number of options (default value).
All (no choice) — SelectAll	All options are selected.
At least one — SelectAtLeastOne	At least one option.

The “Add group” button of the step editor (or the step’s right-click menu) creates a group, of type “Any number” by default; “Delete” in the group’s right-click menu or the group editor removes it.

Reordering — the display order is adjusted without recreating everything: drag a node in the tree and drop it where it should go (an option before another option or onto a group, a group before another group or onto a step, a step before another step), or use “Move before” / “Move after” in the node’s right-click menu. “Duplicate” copies a step, a group or an option with everything it contains. Every change can be undone with Edit → Undo (Ctrl+Z) and redone with Edit → Redo (Ctrl+Y); consecutive keystrokes in a field count as one step. Cut, Copy and Paste — in the same right-click menu, in the Edit menu or with Ctrl+X / Ctrl+C / Ctrl+V — move a node elsewhere in the project or into another open FOMOD (§ 8.30).

Options and default types

An option (a <plugin> element in the FOMOD format) is a choice offered to the user. It has a name, a description, an image (preview, "Browse…" and "Clear" buttons) and a default type:

Type	Meaning
Optional — Optional	Optional option (default).
Required — Required	Option that is always installed.
Recommended — Recommended	Recommended but optional option.
Not usable — NotUsable	Unusable option.
Could be usable — CouldBeUsable	Potentially usable option depending on conditions.

Plugin files

The “Source” section presents, as a four-column table (Type, Source, Destination, Priority), the files and folders installed by the option. The “Add file”, “Add folder” and “Remove” buttons manage the list. The Destination column is editable: click a cell and type the path, relative to the game’s Data folder, where the item must be installed. It is empty by default, which installs the file — or the contents of the folder — at the root of Data; this is the right choice when the source folder already reproduces the game’s structure (for example a folder that itself contains meshes, textures, SKSE, etc.). Enter an explicit path

only when the target differs, for example SKSE\Plugins\MyMod.ini. Files must be located under the root directory. You can also drag files or folders straight from your file explorer (Windows Explorer, Finder…) onto the XIMOD window while the option is selected: they are added to its Source table at once (see § 8.5). The dialog opens in the folder of your previous pick, so sibling folders are one click away (§ 8.28). Rename on disk… renames the selected file or folder in the mod folder and updates every reference to it (§ 8.31).

Group / page destination

To send several options to the same place without editing each file, use the bulk-destination controls. In a group's editor, the "Destination for the whole group" field and its "Apply to all options in this group" button set one install destination on every file of every option in that group at once. At the step level, the collapsible "Install destination (whole page)" panel does the same for every option on the page, across all its groups. A status message confirms how many files were updated. The button stays disabled until the options actually contain files, and you can still fine-tune any individual destination afterwards in the option's Source table. These controls also reach the files the preset wizard (§ 8.29) moved to conditional installs: the conditional sets fed by the options of the group or page are updated too.

Condition flags

The "Flag name:" section lets you associate flags with the option (name = value pairs). When an option is selected by the end user, its flags are set; they can then condition the visibility of a step, the type of an option (dependency pattern) or the installation of conditional files. The "Add flag" and "Remove flag" buttons manage the list.

Autocompletion — the name and value fields of flags and dependencies offer, as you type, a menu of suggestions drawn from the elements already present in the project, which avoids typos that would silently break a match. For a file dependency, the value suggests the states Active / Inactive / Missing. The suggestions are not binding: a new value can still be entered freely.

Option dependencies. Below the flags, this section makes an option's type dynamic. The Default Type above applies when no pattern matches; each pattern gives the option another type when its conditions are met. Use New Pattern / Delete Pattern to manage the list; for the selected pattern choose the Pattern Operator (And / Or) and the Pattern Type to apply (Optional, Required, Recommended, NotUsable or CouldBeUsable), then add its conditions. A condition has a type (File or Flag), a File/Flag name (with autocompletion) and, for a file, a State (Active, Inactive or Missing) or, for a flag, its expected value. For example, set the Default Type to NotUsable and add a pattern of type Optional whose condition is the file DLCCoast.esm in state Active, so the option becomes selectable only when Far Harbor is installed.

Step visibility conditions. Each step can be shown conditionally. Under the step name, the collapsible Visibility Conditions section holds the step's visibility rule: set the Operator (And / Or) and add File or Flag conditions the same way. When it contains at least one condition, the step appears in the wizard only when they are satisfied — for example a patch step shown only when DLCCoast.esm is Active. Leave it empty to always show the step.

4.3 The Required Installs tab

This tab lists the files and folders installed systematically, regardless of the user's choices. It presents the same four-column table (Type, Source, Destination, Priority) — with the same editable Destination column — and the same "Add file", "Add folder" and "Remove" buttons as the files section of an

option. Above the table, the Destination field and its Apply to all required files button set one install destination on every required file at once — the counterpart of the group / page controls of § 4.2; each destination can still be fine-tuned afterwards.

4.4 The Conditional Installs tab

This tab groups patterns: each pattern installs a set of files only when its conditions are met. A row of "Pattern N" tabs lets you add ("➕") or remove ("➖") a pattern.

For the selected pattern:

- **Operator** — And or Or: how the pattern's dependencies are combined.
- **Dependencies** — a list of conditions of type Flag ([Flag] name = value) or File ([File] name (state)). The "Add dependency" and "Remove dependency" buttons manage the list.
- **Files** — the four-column table of files installed when the conditions are satisfied. Above it, the Destination field and its Apply to all files of this set button set one install destination on every file of the pattern at once — handy after the preset wizard (§ 8.29) has moved the options' files here. **At the top of the tab, a Destination field and its Apply to all conditional sets button set one install destination on every file of every set at once — after the preset wizard, there can be one set per option.**

Note — "plugin dependencies". A *plugin dependency* is simply a **File** dependency whose file is a game plugin (.esm/.esp). In the **name** field, enter the plugin's filename as it appears in the game (for example Fallout4.esm, DLCCoast.esm, or another mod's .esp), not the mod's display name. Choose the **state**: *Active* (plugin present and enabled — the usual choice), *Inactive* (present but disabled) or *Missing* (absent). A File dependency can be used in three places: to install files automatically (the **Conditional Installs** tab), to show or hide a wizard step (a step's *visible* condition), or to change an option's state — for example make it *Recommended* when a DLC is active, or *NotUsable* when a required plugin is missing.

4.5 The "Settings" window

Accessible through Tools → Settings (Ctrl+,). It has two tabs: General and Recent Files. The settings are kept in Config.ini.

The **General** tab is organized into two columns.

Left column — the country flag and its name:

Item	Role
Flag	Clicking the image opens the country selector (see below).
Country name	Name of the country written in the selected language (endonym). Non-editable field, shown on two lines.

Right column — the interface settings:

Setting	Description
Language	Interface language, applied immediately. The field stays disabled as long as no country is chosen; it then offers the languages spoken in that country.
Theme	Dark, Light or System.
Font size	Size of the interface text.

Setting	Description
Process line breaks	Interprets \n sequences in descriptions.
Max recent files	Number of entries kept.
Window Width / Height	Stored dimensions of the main window.

Country selector — clicking the flag opens the "Country selection" window, which presents the 244 flags as a grid, with vertical and horizontal scrolling. It is a free window, centred on the main window when it opens (see § 2.6). A "Filter:" field lets you narrow the list by French name, English name or country code; a clear icon (red cross) to the right of the field empties the filter with a single click. The country name appears under each flag.

The grid can also be navigated **from the keyboard**: the arrow keys (or the numeric keypad 8/2/4/6) move the selection frame, Page Up/Page Down scroll one screen, Home/End jump to the first and last flag, Tab and Shift+Tab move forward and back one flag, Enter confirms the highlighted country and Esc closes the window. Details are given in Appendix B.

The choice of country determines the list of languages offered: XIMOD relies on the CountryLanguages.json file, which lists all the languages spoken in each country — up to 844 for Papua New Guinea, hence the scrolling of the drop-down list.

Three settings were added in version 2: Backups to keep (number of previous versions of the FOMOD XML kept under fomod/backups at each save, 0 = none, default 10); Autosave recovery copy every N minutes (a recovery copy of every modified project is written in the configuration folder at this interval and offered on the next start only after an abnormal exit, 0 = off, default 5); and Add the masters of a plugin as conditions (see § 8.17, on by default).

4.6 Pre- and post-save scripts

XIMOD Architect can automatically run a script at save time: a pre-save script, launched just before the FOMOD files are written, and a post-save script, launched just after. They serve to automate the tasks surrounding mod generation — for example cleaning up old archives before saving, incrementing a build number, or automatically packaging the mod into an archive ready to publish once the FOMOD is written.

They are edited through Tools → Pre-Save Script… and Tools → Post-Save Script…. The window shows a help text, the list of available macros, an input area in a fixed-width font, and the "Save" and "Cancel" buttons. The contents of both scripts are kept in Config.ini (keys PreSaveScript and PostSaveScript), and are therefore remembered from one session to the next.

How it works: at save time, XIMOD first replaces the macros with their values, writes the result to a temporary file, then runs it with the system interpreter — a `.bat` command file launched by `cmd` on Windows, a `.sh` shell script launched by `sh` on Linux and macOS. The temporary file is deleted afterwards. The pre-save script runs before the XML files are written, the post-save script just after; a script left empty is simply ignored.

Before execution, the following macros are replaced in the script text:

Macro	Substituted value
$MODNAME$	Mod name.
$MODAUTHOR$	Author name.

Macro	Substituted value
$MODVERSION$	Mod version.
$MODROOT$	Path of the root directory.
$DATE$	Current date (YYYY-MM-DD).
$TIME$	Current time (HH:MM:SS).
$RANDOM$	Random number.

Example — pre-save script (Windows)

Delete the .rar archives already present at the root of the mod, to start from a clean folder before saving:

```
del "$MODROOT$\*.rar" /q
```

Example — post-save script (Windows)

Once the FOMOD is written, automatically package the entire mod content into a .rar archive named after the mod name and version, using WinRAR:

```
d:
cd "$MODROOT$"
"C:\Program Files\WinRAR\rar.exe" a -r -ep1 "$MODNAME$_$MODVERSION$.rar" *
```

Line by line: `d:` switches to drive D:, `cd "$MODROOT$"` moves to the root of the mod, then WinRAR creates the archive (`-r` recursive, `-ep1` so as not to reproduce the parent tree) named, for example, `Dawnlight_1.2.0.rar`.

Linux / macOS equivalent

On Linux and macOS, the script is interpreted as a shell script. The same post-save packaging is written, for example, with zip:

```
cd "$MODROOT$"
zip -r "$MODNAME$_$MODVERSION$.zip" .
```

> **Note —** *These scripts run real system commands with your user rights; a command such as del or rm is destructive. Test them on a throwaway project before using them on a real mod.*

4.7 The "About" window

Opened through Help → About, this window recalls the application name, its version number, a brief description ("A cross-platform FOMOD installer creation tool for mods of any game that Vortex or Mod Organizer 2 can manage.") and the note "Licensed under MIT". An "OK" button closes it.

4.8 Generating the FOMOD files

On save, XIMOD Architect translates the project into two standardized XML files, written in UTF-8 in the fomod subfolder.

Before writing, the project is checked: if it contains anomalies (missing mod name, no step or required file, a step or group without a name, a group without an option), a window lists them all and offers "Save anyway" — saving is never blocked, which lets you deliberately save a work-in-progress project.

Schema validation — on top of this project check, there is a structural check of the generated ModuleConfig.xml against the ModConfig 5.0 schema, written in Rust (without an external XSD

engine): element hierarchy and cardinality, mandatory attributes and enumeration values (selection and plugin types, file states, operators). Each anomaly is located by its line and column. It is accessible in three ways: the Project → Validate FOMOD command, which shows a full report; live in the XML editor as soon as the document is well-formed; and integrated into the save confirmation window.

info.xml contains the metadata, under a `<fomod>` root: the name (Name), the author (Author), the version (Version), the website (Website), the description (Description), the category (Groups) and the game (Game). The Game element is a XIMOD-specific extension, harmlessly ignored by mod managers, that serves to restore the correct list of categories when reloading.

ModuleConfig.xml describes the installation, under a `<config>` root: the module name, the header image, the required files (requiredInstallFiles), the steps (installSteps → group → plugin), and the conditional installs (conditionalFileInstalls). Each plugin describes its description, its image, its condition flags, its files and its type. Dependencies are combined by an And or Or operator.

4.9 Translation editor

Accessible through Tools → Translate the interface…, this tool lets you translate the interface without editing the files by hand, and correct an existing translation.

Window header

Field	Role
Flag	Choice of country, which determines the list of translatable languages.
Country endonym name	Name of the country in the translated language, shown with the chosen font.
Language endonym name	Name of the language in that language itself (e.g. "Français").
Author	Name of the translator.
Displayed language	Reference language shown in column 2 (among those that already have a main.ftl file).
Language to translate	Target language, chosen from the languages of the country.
Font	Font used for this language. The "Browse…" button opens a file selector positioned on assets/fonts.
Google Fonts	Opens the fonts.google.com site in the browser.

Important constraint: the font must be installed in the assets/fonts folder. A font located elsewhere is rejected, because the program loads it via a path relative to that folder and would not find it at the next startup. If the font is missing, you therefore need to download it from Google Fonts and copy it there before selecting it.

Likewise, only a language that has an **ISO 639-3 code** can be translated: it is this code that names the translation folder and links the language to its font and its reference data. The save buttons are disabled otherwise.

Translation table — three columns:

Column 1	Column 2	Column 3
Key (non-editable)	Label in the displayed language	Translation input field

Token protection: the elements that must not be translated (macros such as $MODNAME$, variables such as { $num }) never appear as-is in column 2 and are preserved automatically.

- Token at the beginning or end of the text: hidden, then reinserted on save.
- Token in the middle of the text: replaced by a compact marker [[1]], [[2]]… to keep and reposition.

The application name (key app-title) is deliberately excluded and is not translatable.

Keyboard navigation — the table can be operated without the mouse (see Appendix B): the Up/Down arrows and Page Up/Page Down scroll the rows, Home/End jump to the first and last, and Enter places the cursor in the input field of the highlighted row. Once editing, Tab moves to the next field and Shift+Tab to the previous one, so translations can be entered one after another without leaving the keyboard. The number of rows moved by Page Up/Page Down adapts automatically to the window height.

Saving — the "Save" button writes the file assets/locales/<language>/main.ftl, preceded by a metadata block written as Fluent comments (the file therefore remains perfectly valid):

```
# XIMOD Architect - translation metadata
# @language = fra
# @font = Noto_Sans/static/NotoSans-Regular.ttf
# @langname = Français
# @author = Patrice
```

The header describes only the **language**: it contains no country information, because the same language is spoken in several countries (German in Germany as well as in Switzerland). The country is determined by the chosen flag, and XIMOD handles the country → languages mapping from Countries.json and CountryLanguages.json.

Filling in the endonym fields — when the country (flag) and the language are chosen, XIMOD fills in the two fields:

- **Country endonym name**: always resolved from Countries.json for the selected country; it therefore follows the flag (German shows "Deutschland" for Germany, "Schweiz" for Switzerland).
- **Language endonym name**: taken from the file header (# @langname) when it has one; failing that — an old file without a header, or a not-yet-translated language — it is taken from Languages.json.

The two fields remain editable: the user can make corrections there, which are propagated to the reference files on save.

Saving updates the **reference data**: the endonym and font of the language in Languages.json, and the country endonym in Countries.json (for the country / language pair). If the language did not yet appear there for this country, an entry is created.

Since the header no longer carries the country, XIMOD writes, alongside the main.ftl, a small tab-separated file **countryEndonyms.tsv**, which records the country endonym for the language, one line per country in the form "country code, language code, endonym". It is this file that makes it possible to recover the country endonym when only the language folder is available.

Sending the translation — the "Send…" button builds a ZIP archive containing the translation file (main.ftl, in its language folder) and, if it exists, the accompanying countryEndonyms.tsv file; XIMOD opens the archive folder then prepares a pre-filled email. The language-specific information (language, font, language endonym, author) travels in the header of the main.ftl; the country endonym travels in countryEndonyms.tsv. Neither Languages.json nor Countries.json is attached — only the codes actually

used are transmitted, which respects the terms of use of the ISO 639-3 codes. The user attaches the file and sends it themselves: no data leaves the machine without an explicit action.

> **Note —** *Only send the package when the translation is complete: any key left untranslated is saved with an empty value, which would then be displayed without text in the interface.*

4.10 XML editor

Accessible through Project → XML Editor → info.xml or ModuleConfig.xml, this tool intended for advanced users lets you view and directly modify the generated XML.

It opens in an **independent window**, freely movable and resizable, including on a second screen.

Two modes

- *Read-only* (default): the XML is shown with syntax highlighting and hanging indentation — lines that are too long wrap, aligning under the start of the line, in the manner of Notepad++. The main window remains usable.
- *Edit* ("Modify" button): the text becomes editable and the tabs of the main window are locked, so that a single source is authoritative.

Syntax highlighting: tags, attribute names, quoted values, comments, declaration and text are distinguished by colours suited to the light or dark theme.

Line numbering: a gutter shows the numbers; the continuation lines of a wrap remain blank.

Live validation: while typing, the XML is checked continuously. A green indicator confirms a well-formed document; in case of error, a red message specifies the line and column, and the faulty token is underlined.

Applying changes: the "Apply" button re-parses the text and updates the project. In case of error, the change is refused and the text remains editable. The "Cancel" button regenerates the text from the project.

4.11 Font management

XIMOD Architect ships with a set of Noto fonts covering all the writing systems of the listed languages, placed in assets/fonts.

These fonts are loaded **dynamically**: at any time, only those actually needed are installed, namely the font of the interface language and those of the languages of the selected country. The load stays low — a single font for most countries.

This mechanism guarantees the correct display of non-Latin scripts (Thai, Burmese, Amharic, Hmong, etc.), which would otherwise appear as empty squares. It applies both to the interface and to the "Properties" window and the translation editor: the fonts of every language displayed there are loaded so that no name or label appears as squares.

XIMOD Architect ships with **32 interface translations**, each covering the full set of strings, including non-Latin scripts: Japanese (jpn), Simplified Chinese (zho), Korean (kor) and Russian (rus). English (eng) and French (fra) are the reference translations, maintained by hand; the others come from contributions that native speakers can review through the translation editor.

The font associated with each language is indicated in Languages.json and can be changed from the translation editor.

4.12 Merging two FOMODs

The File → Merge FOMOD… command lets you combine two installers by joining their installation content. The currently open project acts as the recipient; it must therefore have been created or opened beforehand, otherwise the command stays disabled.

A click opens a file selector: choose the ModuleConfig.xml file of the FOMOD to integrate (the donor). Its steps, required files and conditional installs are then added to the end of those of the recipient project. The metadata of the recipient project — name, author, version, category, header image — is preserved: only the installation content is imported.

After the merge, the project is marked as modified: remember to save it (Ctrl+S) to write the result to the FOMOD files. The merge can be repeated to integrate several FOMODs one after another.

> **Note —** *Only the donor's ModuleConfig.xml file is read; its own metadata (info.xml) is not imported. After merging, check that priorities and conditions remain consistent across the combined steps.*

4.13 Exporting the distribution archive

The File → Export distribution archive… command produces, in a single operation, the archive ready to be uploaded (Nexus Mods or any other mod manager). XIMOD first writes the FOMOD (fomod/info.xml and fomod/ModuleConfig.xml) into the root directory, then compresses the whole folder in ZIP format. The command is disabled as long as no root directory is defined.

A selector offers a default name of the form "<name>-<version>.zip" (sanitized of forbidden characters); you can change it before saving. The status bar then indicates the number of files written and the path of the archive.

Archive layout — the fomod/ subfolder and the mod files are at the root of the archive, exactly as mod managers expect. Internal paths use the forward slash "/" (cross-platform compatibility).

> **Note —** *Junk files (.git, .DS_Store, Thumbs.db, desktop.ini…) and the archive itself are automatically excluded. Make sure, therefore, that all the mod files are indeed under the root directory before exporting.*

4.14 The "Properties" window

Opened through Tools → Properties…, this independent window — movable and resizable, including on a second screen — offers a read-only explorer of the embedded country / language database. The database cannot be modified here; it is a consultation tool.

Countries tab: for a chosen country, the window shows its flag, its French and English names, its official languages (with endonym, font and ISO 639-3 / 639-1 codes) and the list of all the languages spoken there.

Languages tab: for a chosen language, the window shows its ISO codes, its font, and performs the reverse lookup of the countries where this language is spoken. The "Spoken in" list names each country in the language you chose in Settings (flag → country → language): its endonym in that language from Countries.json when the language is official there ("Confederazione Svizzera" for an Italian-speaking user), otherwise the translated name from CountryNames.json ("Südafrika" for a German-speaking user), and the English name as a last resort.

Keyboard navigation — both the country list and the language list can be operated from the keyboard (see Appendix B): Up/Down arrows, Page Up/Page Down (whose step adjusts to the window height), Home/End and Enter to select the highlighted item. Tab and Shift+Tab move the selection within the list

without ever landing on the flag. Each "Filter:" field has a clear icon (red cross) to its right, and the window closes with Esc.

4.15 Command-line mode (CLI)

Launched with arguments, XIMOD Architect opens no window: it runs the requested command then terminates with an exit code (0 on success, non-zero on failure or validation anomaly). This mode makes it possible to automate the rebuilding and packaging of one or more FOMODs in a script or a continuous-integration pipeline.

> **Note —** *On Windows, since the executable is compiled as a "windows" application, it attaches to the console that launched it (via AttachConsole) to write its messages there. Launch it from a command prompt (cmd or PowerShell) to see the output; double-clicking a CLI command would display nothing.*

The available commands are:

Command	Effect
ximod-architect validate <root>	Validates the FOMOD of the directory (project checks and conformity to the ModConfig 5.0 schema); each anomaly is located. Non-zero exit code if an anomaly is found.
ximod-architect build <root>	(Re)writes fomod/info.xml and fomod/ModuleConfig.xml from the existing project.
ximod-architect package <root> [-o FILE]	Writes the FOMOD then builds the .zip distribution archive. The -o option lets you specify the output path; failing that, the archive is created in the current directory.
ximod-architect help	Displays the help (list of commands).
ximod-architect version	Displays the version number.

Here, `<root>` denotes the root directory of the mod (the folder that contains — or will contain — the fomod subfolder).

4.16 Checking for updates

By default, XIMOD Architect checks once a day, when it starts, whether a newer version is available. If one is, a discreet banner appears below the menu bar with links to download it (Nexus and GitHub); you can also check at any time from Help → Check for updates. This can be turned off in the Settings window (“Check for updates on startup”).
Privacy: this check is the only network request the application makes. It sends no personal data — only a single secure (HTTPS) request to the public GitHub releases page to read the latest version number. XIMOD never downloads or installs anything by itself; you always choose whether to follow the download link.

5. FOMOD tag reference

This section describes the tags actually produced by XIMOD Architect. The two files are written in UTF-8 (with a byte order mark) in the mod's fomod subfolder: info.xml is indented in four-space steps, ModuleConfig.xml with a tab. When reading, the encoding is detected automatically thanks to the byte order mark, which makes it possible to open both UTF-8 and UTF-16 (LE or BE) files produced by other tools.

5.1 The info.xml file

The metadata, under a `<fomod>` root. The elements are written in the order of the table below; those marked "if not empty" are omitted when the corresponding field is left empty.

Tag	Content	Written
`<Name>`	Mod name.	Always
`<Author>`	Author.	If not empty
`<Version>`	Version.	If not empty
`<Website>`	URL of the mod's page.	If not empty
`<Description>`	Description.	If not empty
`<Groups>`	Category of the mod.	Always
`<Game>`	Identifier of the selected game. XIMOD-specific extension (outside the FOMOD specification), ignored by mod managers.	If not empty

Complete example:

```
<?xml version="1.0" encoding="utf-8"?>
<fomod>
    <Name>My Awesome Mod</Name>
    <Author>Patrice</Author>
    <Version>1.0.0</Version>
    <Website>https://www.nexusmods.com/skyrimspecialedition/mods/12345</Website>
    <Description>A sample mod for the manual.</Description>
    <Groups>Player homes</Groups>
    <Game>skyrimSpecialEdition</Game>
</fomod>
```

5.2 The ModuleConfig.xml file

The installation logic, under a `<config>` root that references the ModConfig5.0.xsd schema. The blocks appear in the following order: module name, header image, required files, install steps, conditional installs.

Root and header

Tag / Attribute	Role
`<config>`	Root. Carries the xmlns:xsi and xsi:noNamespaceSchemaLocation attributes (ModConfig5.0.xsd schema).
`<moduleName>`	Module name, shown at the top of the wizard.
`<moduleImage path="…"/>`	Header image (empty tag, written only if an image is defined).

Files and folders

Tag / Attribute	Role
`<file source="…" destination="…" priority="…"/>`	Copy of a file. source = path in the archive (relative to the root); destination = target under Data; priority = installation order.
`<folder source="…" destination="…" priority="…"/>`	Copy of a whole folder (same attributes).
`<requiredInstallFiles>`	Container of the files installed systematically.
`<files>`	Container of the files tied to an option or a conditional pattern.

Steps and groups

Tag / Attribute	Role
`<installSteps order="Explicit">`	Container of the steps (the defined order is preserved).
`<installStep name="…">`	A page of the wizard.
`<visible>`	Visibility conditions of the step (contains <dependencies>).
`<optionalFileGroups>`	Container of the option groups of the step.
`<group name="…" type="…">`	A group; type = selection rule (see below).
`<plugins order="Explicit">`	Container of the options of the group.

Values of the `type` attribute of a `<group>`:

type	Constraint
Exactly one (required) — SelectExactlyOne	Exactly one option.
At most one — SelectAtMostOne	At most one option.
Any number — SelectAny	Any number.
All (no choice) — SelectAll	All, enforced.
At least one — SelectAtLeastOne	At least one.

Options (plugins)

Tag / Attribute	Role
`<plugin name="…">`	An option offered to the user.
`<description>`	Descriptive text of the option (always written).
`<image path="…"/>`	Image illustrating the option (if defined).
`<conditionFlags>`	Container of the flags set if the option is chosen.
`<flag name="…">value</flag>`	A flag: name in the attribute, value in the content.
`<typeDescriptor>`	Describes the type of the option (simple or conditional).
`<type name="…"/>`	Simple type (when no dependency pattern is defined).
`<dependencyType>`	Dynamic type (when patterns exist).
`<defaultType name="…"/>`	Default type applied if no pattern applies.

Values of a <type> / <defaultType>:

Type	Effect in the wizard
Optional — Optional	Option unchecked, freely activatable.
Required — Required	Mandatory option, cannot be deselected.
Recommended — Recommended	Option checked by default, can be deselected.
Not usable — NotUsable	Option disabled (cannot be selected).
Could be usable — CouldBeUsable	Usable, but flagged as potentially risky.

Dependencies and conditions

Tag / Attribute	Role
<dependencies operator="…">	Group of conditions; operator = And or Or.
<fileDependency file="…" state="…"/>	Condition on a file; state = Active, Inactive or Missing.
<flagDependency flag="…" value="…"/>	Condition on a flag and its expected value.
<conditionalFileInstalls>	Section of the conditional installs.
<patterns> / <pattern>	List of patterns; each pattern associates <dependencies> with <files> (or, within a typeDescriptor, with a <type>).

6. The ModuleConfig.xml file explained in detail

Chapter 5 presented the tags as reference tables. This chapter goes through them one by one, in plain language, to explain what each one does, what options it accepts, and above all what concrete effect these options have on the installation wizard that the mod's user will see. No prior knowledge of XML is required: XIMOD Architect writes this file for you; understanding it only helps you design more reliable installers.

A few words of vocabulary. An XML file is made of "tags" enclosed in angle brackets. An opening tag `<step>` goes together with a closing tag `</step>`; whatever is written between the two belongs to that tag. A tag can carry "attributes", that is, settings written in the form `name="value"`. Thus `<group name="Resolution" type="SelectExactlyOne">` is a "group" tag with two attributes. Finally, a tag with no content closes on itself, like `<file … />` (note the trailing /).

6.1 How the installer reads the file

When the user installs your mod, their manager (Vortex, MO2, NMM) reads ModuleConfig.xml from top to bottom and builds a wizard: a series of pages (the steps), each offering groups of options to check. At the end, the manager gathers all the files matching the chosen options and copies them into the game. Three ideas to keep in mind:

- **order matters**: the steps are displayed in the order they appear in the file;
- **checking installs nothing right away**: checking an option only records a choice; the actual copying of files happens at the very end;
- **conditions can modulate everything**: hiding a step, changing the state of an option or triggering the installation of additional files — this is the role of flags and dependencies (§ 6.6).

6.2 The root and the header

`<config>` — the root. The whole configuration fits inside a single `<config>…</config>` tag. It carries two technical attributes (xmlns:xsi and xsi:noNamespaceSchemaLocation) that designate the validation schema (ModConfig 5.0). You never have to touch them: XIMOD always writes them identically.

`<moduleName>` — the mod name, shown at the top of the wizard. It is the one entered in the Mod Info tab.

`<moduleImage path="…"/>` — the banner image shown at the top of the wizard. The path attribute indicates its location, relative to the root of the mod (e.g. fomod\header.png). The tag is empty (it closes on itself). Without an image, it is simply absent.

6.3 Installing files

Two tags describe what will be copied into the game: `<file>` copies a single file, `<folder>` copies a whole folder with all its content. Both accept the same three attributes:

Attribute	Role	Example
`source`	Path of the element in your mod's archive, relative to the root.	`tex\2k`
`destination`	Where the element must land, relative to the game's Data folder.	`textures`

Attribute	Role	Example
priority	Installation order in case of conflict (see below).	0

The priority attribute and its impact. It is a number that sets the copy order when two options install a file in the same place. The rule: low priorities are installed first, high ones afterwards — and since the last written wins, a higher priority "overwrites" a lower one. Example: if a base option installs a texture at priority="0" and a "high resolution" option installs the same texture at priority="1", it is the high-resolution version that remains, regardless of the display order of the options. Leave 0 everywhere as long as you have no deliberate conflict to arbitrate.

> **Note —** *The destination is empty by default, which installs the item at the root of the game's Data folder. Click the Destination cell to enter an explicit path (relative to Data) whenever the item must go elsewhere. An empty destination is the correct choice when the source folder already mirrors the game's structure.*

These tags appear in three places: in <requiredInstallFiles> (files always installed), in the <files> of an option (installed if the option is chosen), and in the <files> of a conditional pattern (installed if conditions are met).

6.4 Steps, groups and options

This is the heart of the wizard. The hierarchy is simple: a step contains groups, a group contains options.

<installSteps order="Explicit"> gathers all the steps; order="Explicit" means "respect the order I wrote". Each page of the wizard is an <installStep name="…">, whose name attribute is the title shown at the top of the page.

<visible> — show the step conditionally. A step can be shown only if certain conditions are true; you place them in a <visible> tag (see § 6.6). Without <visible>, the step is always displayed. Example: show an "advanced settings" step only if the user checked "expert installation" earlier.

<optionalFileGroups> contains the groups of the step. Each <group name="…" type="…"> has a name (section title) and, above all, a type that sets the selection rule — that is, what the user is allowed to check:

type	What the user sees and can do
Exactly one (required) — SelectExactlyOne	Radio buttons: they must choose one option, and only one. Neither zero nor two. Ideal for exclusive variants (a single texture resolution).
At most one — SelectAtMostOne	As above, but they can also choose nothing: an "option or nothing".
Any number — SelectAny	Independent checkboxes: they check whatever they want, from none to all. The most common case, for stackable modules.
At least one — SelectAtLeastOne	Checkboxes, but they must check at least one to continue. Prevents leaving empty-handed.
All (no choice) — SelectAll	All options are checked automatically and locked: the user cannot uncheck them. Used to present mandatory files while describing them.

In a group, <plugins order="Explicit"> contains the options, each being a <plugin name="…"> (the word "plugin" here means an option of the wizard, not an .esp file). Its name is the label the user checks. An option is made up of:

- <description>: the text shown on the right when the option is selected — describe its effect clearly there;

- `<image path="…"/>`: an illustration shown with the description (optional);
- `<conditionFlags>`: the flags set when the option is chosen (§ 6.6);
- `<files>`: the files installed if the option is chosen;
- `<typeDescriptor>`: the default state of the option (§ 6.5).

6.5 An option's type and its behaviour

Each option has a "type" that determines its default state in the wizard: checked or not, editable or not. It is described by `<typeDescriptor>`. In the simple case, it contains a single `<type name="…"/>`. The five values and their effect:

type	Effect in the wizard
Optional — Optional	Unchecked by default, freely checkable and uncheckable. The common case.
Recommended — Recommended	Checked by default, but the user can uncheck it. For an advised choice.
Required — Required	Checked and locked: cannot be unchecked. For an essential component.
Not usable — NotUsable	Unchecked and locked: cannot be checked. Indicates an option unavailable in the current context.
Could be usable — CouldBeUsable	Checkable, but flagged as potentially problematic (often with a warning).

Dynamic types: `<dependencyType>`. An option's type can also change depending on conditions. Instead of a fixed `<type>`, you write a `<dependencyType>` that contains a `<defaultType name="…"/>` (the default type, when no condition is met) and a list of `<pattern>`: each pattern associates conditions with a `<type>` to apply if they are true.

Example: an "ENB Ultra preset" option can be Optional by default, become Recommended if the user enabled dynamic candles, and become NotUsable if a required file is missing on disk. It is this mechanism that makes an installer "smart". A complete example appears in Appendix D. Section 7.5 uses it to turn a "preset" into defaults the user can still adjust.

6.6 Flags, dependencies and conditional installs

These three notions form the FOMOD condition system — what allows an installer to react to the user's choices.

Flags — `<conditionFlags>` and `<flag>`. A flag is a small variable that you set when an option is chosen. An option's `<conditionFlags>` tag contains one or more `<flag name="…">value</flag>`: name is the flag name, and the text between the tags is its value. Thus `<flag name="res">4K</flag>` sets the "res" flag to "4K" as soon as the option is checked. A flag has no visible effect on its own: it serves as memory, reused later as a condition elsewhere.

Dependencies — `<dependencies operator="…">`. A dependency is a condition to check. They are grouped in `<dependencies>`, whose operator attribute says how to combine them: `And` (all conditions must be true) or `Or` (a single one is enough). Two kinds of conditions exist:

Condition	What it tests

Condition	What it tests
`<flagDependency flag="…" value="…"/>`	True if the named flag has exactly the indicated value. This is how you react to choices made earlier (e.g.: the "res" flag equals "4K").
`<fileDependency file="…" state="…"/>`	True depending on the state of a file already present, or not, in the user's game.

The `state` attribute of a file dependency takes three values:

state	Meaning
Active	The file is present and enabled in the game.
Inactive	The file is present but disabled.
Missing	The file is absent.

These dependencies serve three things: the visibility of a step (in `<visible>`), the dynamic type of an option (in a `<pattern>` of `<dependencyType>`), and the conditional installs below.

Conditional installs — `<conditionalFileInstalls>`. This section, placed after the steps, installs files only when conditions are met, without the user having to check anything directly. It contains `<pattern>` elements, each associating `<dependencies>` with a `<files>`. If the pattern's conditions are true at the end of the wizard, its files are installed.

Difference with step visibility: `<visible>` decides whether a page is shown, whereas `<conditionalFileInstalls>` decides whether files are installed, behind the scenes. Typical case: automatically install a compatibility patch only if the user checked two particular options (whose flags are then both set).

> **Note —** *A flag must first be set by an option (via <conditionFlags>) before it can be tested by a dependency. So define your flags in the steps, then test them in the conditional patterns.*
>
> **Note —** *Design rule: when an option only stands for a decision — a preset, a profile, a "module" switch — let it set flags and nothing else, and leave the copying to conditional installs keyed on those flags. One flag per feature is then the single source of truth: whatever the combination of choices, a preset and a hand-picked option can never install the same files twice. Section 7.5 applies this rule.*

7. ModuleConfig.xml examples

The examples below illustrate the most common cases. They correspond to the XML actually produced by XIMOD Architect (indentation simplified for readability).

7.1 Minimal example — required files

A mod with no choices: all the files are installed directly.

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>My Simple Mod</moduleName>
    <requiredInstallFiles>
        <file source="MyMod.esp" destination="MyMod.esp" priority="0"/>
        <folder source="textures" destination="textures" priority="0"/>
    </requiredInstallFiles>
</config>
```

7.2 Exclusive choice — SelectExactlyOne

A step offering the choice of a single texture resolution; the 2K option is recommended by default.

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>HD Textures</moduleName>
    <moduleImage path="fomod\header.png"/>
    <installSteps order="Explicit">
        <installStep name="Texture resolution">
            <optionalFileGroups>
                <group name="Resolution" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="2K">
                            <description>2K textures (balanced).</description>
                            <files>
                                <folder source="2k" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Recommended"/>
                            </typeDescriptor>
                        </plugin>
                        <plugin name="4K">
                            <description>4K textures (high quality).</description>
                            <files>
                                <folder source="4k" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Optional"/>
                            </typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
    </installSteps>
</config>
```

7.3 Conditional install — flags

An option sets a flag when it is checked; a file is installed only if that flag is active.

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>Conditional patch</moduleName>
    <installSteps order="Explicit">
        <installStep name="Options">
            <optionalFileGroups>
                <group name="Modules" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="Module A">
                            <description>Enables module A.</description>
                            <conditionFlags>
                                <flag name="ModuleA">On</flag>
                            </conditionFlags>
                            <files>
                                <folder source="A" destination="meshes" priority="0"/>
                            </files>
                            <typeDescriptor>
                                <type name="Optional"/>
                            </typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
    </installSteps>
    <conditionalFileInstalls>
        <patterns>
            <pattern>
                <dependencies operator="And">
                    <flagDependency flag="ModuleA" value="On"/>
                </dependencies>
                <files>
                    <file source="patchA.esp" destination="patchA.esp" priority="0"/>
                </files>
            </pattern>
        </patterns>
    </conditionalFileInstalls>
</config>
```

7.4 One page, several steps — SelectAny + flags

A single multiple-choice step can open a different step for each option the player ticks: steps are not tied to options. Each option sets a condition flag, and each target step carries a visibility condition on that flag. Ticking A and B then shows steps A and B; ticking A, B and C shows three steps.

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>Modular Add-ons</moduleName>
    <installSteps order="Explicit">
        <!-- Step 1: the choices. Each ticked option sets its own flag. -->
        <installStep name="Choose modules">
            <optionalFileGroups>
                <group name="Modules" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="Module A">
                            <description>Enable module A.</description>
                            <conditionFlags>
                                <flag name="want_A">on</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <plugin name="Module B">
                            <description>Enable module B.</description>
                            <conditionFlags>
                                <flag name="want_B">on</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <!-- Modules C and D follow the same pattern (want_C, want_D). -->
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Step A: shown only when want_A is on. -->
        <installStep name="Module A options">
            <visible>
                <flagDependency flag="want_A" value="on"/>
            </visible>
            <optionalFileGroups>
                <group name="Module A" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="A - extra textures">
                            <description>Optional high-res textures for A.</description>
                            <files>
                                <folder source="a_textures" destination="textures" priority="0"/>
                            </files>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Step B: shown only when want_B is on. -->
        <installStep name="Module B options">
            <visible>
                <flagDependency flag="want_B" value="on"/>
            </visible>
            <optionalFileGroups>
                <group name="Module B" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="B - extra meshes">
                            <description>Optional meshes for B.</description>
                            <files>
                                <folder source="b_meshes" destination="meshes" priority="0"/>
                            </files>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Steps for C and D have exactly the same shape, each guarded by its
             own <visible><flagDependency flag="want_C | want_D" value="on"/>. -->
    </installSteps></config>
```

7.5 Preset with customisable defaults

A “preset” is an option that stands for several choices at once. Users want both: pick the preset and be done, or pick the preset and still adjust one or two features. The key is to separate deciding from installing. The first page decides — it sets flags. The feature pages, shown on demand, let the user choose by hand — they set other flags. The conditional installs do the copying, from those flags.

Step 1 offers the preset and, for each feature, a two-option group: Use the preset’s choices or Customise. The second option opens the page of that feature, which is visible only when its Customise flag is on (§ 7.4). On that page the options use a dynamic type (§ 6.5): the variant the chosen preset would have installed is Recommended — pre-ticked — and None is recommended otherwise, so the user can keep the preset’s variant, switch to another one, or remove the feature.

Neither the preset nor the page options carry any <files>. All copying happens in <conditionalFileInstalls>, one pattern per option: its files are installed when the feature is customised and the option is ticked on its page, or when the feature is left to the preset and that preset chose the option. This is what makes double installs impossible: for each feature, exactly one of the two branches applies, hence one set of files.

> **Note —** *One flag, one writer. Mod managers keep the flags in a table updated at every click: selecting an option writes its flags, deselecting it removes them, and the last writer wins. If two options that can be selected together write the same flag — a preset that also sets a page’s flags, a Customise checkbox that resets them — going back and clicking one of them again silently overrides what the other still shows as selected. So let each flag be written by one group of mutually exclusive options only: preset by the Preset group, cust_A by the mode group of feature A, feat_A by the options of page A. The flags then always match what the user sees, whatever the order of the clicks.*

```
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>Preset with customisable defaults</moduleName>
    <installSteps order="Explicit">
        <!-- Step 1: the preset writes the preset flag, and nothing else. -->
        <installStep name="Installation mode">
            <optionalFileGroups>
                <group name="Preset" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="Preset A (recommended set-up)">
                            <description>Installs feature A in version 2 and feature B in version
1.</description>
                            <conditionFlags><flag name="preset">A</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <plugin name="No preset (build my own)">
                            <description>Choose every feature by hand.</description>
                            <conditionFlags><flag name="preset">none</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
                <!-- One group per feature: it alone writes the feature's Customise flag. -->
                <group name="Feature A" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="Use the preset's choices">
                            <description>Feature A as the preset installs it.</description>
                            <conditionFlags><flag name="cust_A">off</flag></conditionFlags>
                            <typeDescriptor><type name="Recommended"/></typeDescriptor>
                        </plugin>
                        <plugin name="Customise">
                            <description>Opens the page of feature A.</description>
                            <conditionFlags><flag name="cust_A">on</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
```

```
            <!-- "Feature B", "Feature C"... have the same two options and write cust_B, cust_C. -->
        </optionalFileGroups>
    </installStep>
    <!-- Step 2: feature A, shown only when the user asked to customise it. -->
    <installStep name="Feature A">
        <visible><flagDependency flag="cust_A" value="on"/></visible>
        <optionalFileGroups>
            <group name="Feature A" type="SelectExactlyOne">
                <plugins order="Explicit">
                    <plugin name="None">
                        <description>Do not install feature A.</description>
                        <conditionFlags><flag name="feat_A">none</flag></conditionFlags>
                        <typeDescriptor>
                            <dependencyType>
                                <defaultType name="Recommended"/>
                                <patterns>
                                    <pattern>
                                        <dependencies operator="And">
                                            <flagDependency flag="preset" value="A"/>
                                        </dependencies>
                                        <type name="Optional"/>
                                    </pattern>
                                </patterns>
                            </dependencyType>
                        </typeDescriptor>
                    </plugin>
                    <plugin name="Version 1">
                        <description>Feature A, version 1.</description>
                        <conditionFlags><flag name="feat_A">v1</flag></conditionFlags>
                        <typeDescriptor><type name="Optional"/></typeDescriptor>
                    </plugin>
                    <plugin name="Version 2">
                        <description>Feature A, version 2 (the preset's choice).</description>
                        <conditionFlags><flag name="feat_A">v2</flag></conditionFlags>
                        <typeDescriptor>
                            <dependencyType>
                                <defaultType name="Optional"/>
                                <patterns>
                                    <pattern>
                                        <dependencies operator="And">
                                            <flagDependency flag="preset" value="A"/>
                                        </dependencies>
                                        <type name="Recommended"/>
                                    </pattern>
                                </patterns>
                            </dependencyType>
                        </typeDescriptor>
                    </plugin>
                </plugins>
            </group>
        </optionalFileGroups>
    </installStep>
    <!-- The pages of features B and C have exactly the same shape. -->
</installSteps>
<!-- Files are copied here only: by the page when the feature is customised, by the preset otherwise.
-->
<conditionalFileInstalls>
    <patterns>
        <pattern>
            <dependencies operator="And">
                <flagDependency flag="cust_A" value="on"/>
                <flagDependency flag="feat_A" value="v1"/>
            </dependencies>
            <files><folder source="FeatureA\v1" destination="" priority="0"/></files>
        </pattern>
        <pattern>
            <dependencies operator="Or">
                <dependencies operator="And">
                    <flagDependency flag="cust_A" value="on"/>
                    <flagDependency flag="feat_A" value="v2"/>
                </dependencies>
                <dependencies operator="And">
                    <flagDependency flag="cust_A" value="off"/>
                    <flagDependency flag="preset" value="A"/>
                </dependencies>
            </dependencies>
            <files><folder source="FeatureA\v2" destination="" priority="0"/></files>
```

```
            </pattern>
            <!-- The same for feature B: v1 is installed by the page or by preset A. -->
        </patterns>
    </conditionalFileInstalls>
</config>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
    <moduleName>Preset with customisable defaults</moduleName>
    <installSteps order="Explicit">
        <!-- Step 1: the preset sets the flags of every feature it contains, and nothing else. -->
        <installStep name="Installation mode">
            <optionalFileGroups>
                <group name="Preset" type="SelectAtMostOne">
                    <plugins order="Explicit">
                        <plugin name="Preset A (recommended set-up)">
                            <description>Installs feature A in version 2 and feature B in version
1.</description>
                            <conditionFlags>
                                <flag name="preset">on</flag>
                                <flag name="feat_A">v2</flag>
                                <flag name="feat_B">v1</flag>
                            </conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                    </plugins>
                </group>
                <group name="Customise" type="SelectAny">
                    <plugins order="Explicit">
                        <plugin name="Customise feature A">
                            <description>Opens the page of feature A.</description>
                            <conditionFlags><flag name="cust_A">on</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <!-- "Customise feature B", "Customise feature C"... set cust_B, cust_C. -->
                    </plugins>
                </group>
            </optionalFileGroups>
        </installStep>
        <!-- Step 2: feature A, shown only when the user asked to customise it. -->
        <installStep name="Feature A">
            <visible><flagDependency flag="cust_A" value="on"/></visible>
            <optionalFileGroups>
                <group name="Feature A" type="SelectExactlyOne">
                    <plugins order="Explicit">
                        <plugin name="None">
                            <description>Do not install feature A.</description>
                            <conditionFlags><flag name="feat_A">none</flag></conditionFlags>
                            <typeDescriptor>
                                <dependencyType>
                                    <defaultType name="Recommended"/>
                                    <patterns>
                                        <pattern>
                                            <dependencies operator="And">
                                                <flagDependency flag="preset" value="on"/>
                                            </dependencies>
                                            <type name="Optional"/>
                                        </pattern>
                                    </patterns>
                                </dependencyType>
                            </typeDescriptor>
                        </plugin>
                        <plugin name="Version 1">
                            <description>Feature A, version 1.</description>
                            <conditionFlags><flag name="feat_A">v1</flag></conditionFlags>
                            <typeDescriptor><type name="Optional"/></typeDescriptor>
                        </plugin>
                        <plugin name="Version 2">
                            <description>Feature A, version 2 (the preset's choice).</description>
                            <conditionFlags><flag name="feat_A">v2</flag></conditionFlags>
                            <typeDescriptor>
                                <dependencyType>
                                    <defaultType name="Optional"/>
                                    <patterns>
                                        <pattern>
                                            <dependencies operator="And">
                                                <flagDependency flag="preset" value="on"/>
                                            </dependencies>
```

```
                                        <type name="Recommended"/>
                                    </pattern>
                                </patterns>
                            </dependencyType>
                        </typeDescriptor>
                    </plugin>
                </plugins>
            </group>
        </optionalFileGroups>
    </installStep>
    <!-- The pages of features B and C have exactly the same shape. -->
</installSteps>
<!-- Files are copied here only, from the final value of each feature flag. -->
<conditionalFileInstalls>
    <patterns>
        <pattern>
            <dependencies operator="And"><flagDependency flag="feat_A" value="v1"/></dependencies>
            <files><folder source="FeatureA\v1" destination="" priority="0"/></files>
        </pattern>
        <pattern>
            <dependencies operator="And"><flagDependency flag="feat_A" value="v2"/></dependencies>
            <files><folder source="FeatureA\v2" destination="" priority="0"/></files>
        </pattern>
        <pattern>
            <dependencies operator="And"><flagDependency flag="feat_B" value="v1"/></dependencies>
            <files><folder source="FeatureB\v1" destination="" priority="0"/></files>
        </pattern>
    </patterns>
</conditionalFileInstalls>
</config>
```

Project → Preview installer… lets you walk through each combination — preset alone, preset plus Customise feature A, Customise feature A alone — and check the final file list before publishing (§ 4.11, § 8.19). Project → Validate FOMOD reports conflicting destinations and recognises this preset / customise pattern (§ 8.1).

Going back in the installer. Mod managers keep the choices of a page the user has already seen: if the user returns to the first page and picks another preset, a feature page already visited keeps its earlier ticks instead of the new preset's defaults. With one writer per flag this is harmless: the flags always match the screen, so what the user finally sees — the preset for the features left to it, the ticks on the customised pages — is exactly what gets installed. The preset wizard (§ 8.29) builds this structure; a structure built by an earlier version, where the preset also wrote the pages' flags, is rebuilt by running the wizard again.

7.6 Introduction and closing pages

The wizard is drawn by the mod manager, not by the installer. An author controls the names, descriptions and images of the steps, groups and options — and nothing else: no fonts, no colours, no layout, no free text block. Within those limits a welcome page and a closing page are easy to obtain.

Welcome page — make the first step a single group of type SelectAll holding one Required option. Its description carries the welcome text, which the manager shows in the description area; its image carries the visual — a banner, a logo, or text rendered as a picture in any typeface you like. Give it no files, or the mod's core files if you want them listed there.

Closing page — the same construction as the last step: a Required option with a farewell message and, if useful, what to do next (load order, patches to install). An installer cannot display a summary of the user's selections: the format gives a page no access to the choices made on the others. The manager shows its own list of the files about to be installed, and the installer preview (§ 4.11) gives you, the author, that summary for every scenario.

```
<installStep name="Welcome">
    <optionalFileGroups>
        <group name="About this installer" type="SelectAll">
```

```
            <plugins order="Explicit">
                <plugin name="Welcome to Dawnlight">
                    <description>Thank you for downloading Dawnlight. The next pages let you choose the
lighting preset, the optional modules and the compatibility patches. Nothing is installed until the last
page.</description>
                    <image path="fomod\welcome.png"/>
                    <typeDescriptor><type name="Required"/></typeDescriptor>
                </plugin>
            </plugins>
        </group>
    </optionalFileGroups>
</installStep>
<!-- ... the real steps ... -->
<installStep name="All done">
    <optionalFileGroups>
        <group name="Before you play" type="SelectAll">
            <plugins order="Explicit">
                <plugin name="Thank you!">
                    <description>Place Dawnlight.esp after your weather mods. Report any problem on the
Nexus page. Enjoy!</description>
                    <image path="fomod\thanks.png"/>
                    <typeDescriptor><type name="Required"/></typeDescriptor>
                </plugin>
            </plugins>
        </group>
    </optionalFileGroups>
</installStep>
```

Save the welcome step as a template (Project → Templates…, § 8.3) to reuse it in every mod.

8. Version 2 — new features

The features below were introduced by version 2.0.0; sections 8.26 to 8.29 by version 2.1.0. They are presented in the order they were added; the numbered sections of chapters 2 to 7 already reflect them. Sections 8.30 to 8.32 were added by version 2.2.0.

8.1 Destination-conflict detection

When you validate a project (Project → Validate FOMOD), XIMOD now also reports options that install different files to the same destination path, which would overwrite each other. Collisions that can happen together are flagged as certain; those that only arise across mutually exclusive options are flagged as possible.
A conflict is only reported as certain when the two options can really be selected together. Besides the exclusive groups (SelectExactlyOne / SelectAtMostOne), XIMOD now follows the flags: if an option sets a flag to one value and the other option sits on a page that is only shown for another value of that flag — and nothing selectable alongside the first option can set that other value — the pair cannot coexist and the conflict is listed as potential only. Typical case: a “preset” option and the matching option of a “choose your own” page.

8.2 7z export

The Export command now offers 7-Zip (.7z) in addition to ZIP. Pick the format in the save dialog; 7z usually produces smaller downloads.

8.3 Reusable templates

Project → Templates… lets you save the selected step as a reusable template and insert saved templates (step, group or option) into any project. Templates are stored under the application's configuration folder.

8.4 New from folder

File → New from folder… scans a folder and proposes a skeleton where each immediate subfolder becomes one option; an obvious header image at the root is detected automatically.

8.5 Drag-and-drop file assignment

With an option selected in the tree, drag files or folders from your file explorer (Windows Explorer, Finder, your Linux file manager) onto the XIMOD window: they are added to the option’s Source table, exactly as with Add file / Add folder, and several items can be dropped at once. Only paths under the project’s root folder are accepted; the others are ignored. For a large installer this is the fastest way to assign sources: open the feature folder once in the explorer and drop each subfolder onto its option.

8.6 FOMOD comparison

Project → Compare with… compares the current project against another FOMOD and lists what was added, removed or changed (steps, options, files and metadata) — handy for writing a changelog.

8.7 Visual condition editor

Project → Condition editor… shows, for every condition flag, which options set it and which step-visibility rules, conditional installs and dependency patterns use it. Flags that are set but never used (or used but never set) are highlighted.

8.8 Image checks and optimization

Validation now warns about header/option images that are too large, in an unsupported format, or unreadable. In the Mod Info tab, Optimize image downscales the header image to sensible bounds.

8.9 Command-line additions

The command line gains: inspect <plugin.esp> (print a plugin's masters, author and ESL flag); batch <manifest.json> (validate, build and package several FOMODs, for continuous integration); and ba2 <dir> <out.ba2> [game] to pack a folder into a general BA2 archive. BA2 packaging is experimental — verify the archive in-game before shipping.

8.10 Background export and validation

Exporting the distribution archive no longer freezes the window: once the FOMOD is written, the compression runs in the background and a progress window shows the number of files processed and the file being added. A Cancel button stops the export and removes the partial archive. Likewise, the disk checks of the validation (missing files, destination conflicts, images) run in the background; the report window shows “Checking files on disk…” until they complete, and warns you if the project changed in the meantime.

8.11 Crash recovery

If XIMOD Architect ever closes unexpectedly, every project with unsaved changes is written to a recovery folder under the application's configuration directory (crash_recovery/<date>/<mod name>/fomod/). At the next start, the status bar shows where the backup was saved; open that folder with File → Open Folder… to get your work back.

8.12 A clearer interface

Several everyday annoyances were addressed. Saving a project that has no folder yet now asks where to save it instead of doing nothing, and File → Save as… (Ctrl+Shift+S) moves a project to another folder. The window title shows the mod name and a ● dot while there are unsaved changes. Successes, warnings and errors appear as notifications in the top-right corner (errors stay until closed), while the status bar keeps the last message and the number of steps and options (the root folder is shown in the Workspace section of the inspector, and the mod name in the window title and the document tab). Choosing a file outside the root folder is now reported instead of being silently ignored.
Terminology — the interface now consistently says “option” for what the FOMOD format calls a plugin (the choices offered in a group), in every language; “plugin” is kept for game plugin files (.esp, .esm, .esl). The theme setting “System” follows the operating system live: switching the OS between light and dark mode switches XIMOD at once, without restarting, and the chosen font size is kept in both. Finally, the datasets (games, countries, languages) are read while the splash screen is displayed rather than after it, so the main window appears sooner.

8.13 Translating an existing FOMOD

Project → Translate a FOMOD… opens a dedicated window to translate the user-facing strings of an installer into another language: the mod name and description, the names of steps, groups and options, and the option descriptions. Flags, file paths and types are never touched, since mod managers compare them literally. Load the active project (it must be saved) or any mod folder with Open a mod folder…; choose the source and target languages, then fill the Translation column, either directly in the table for names or in the lower panel for descriptions (the source is shown side by side, with spaces and line breaks optionally made visible). Filters (untranslated, to review, with problems, locked), a search box, a progress bar and Ctrl+Enter (next untranslated) help keep track. Author and website are locked by default: locked strings are written unchanged.

Problems are flagged on each row: protected tokens (file names, URLs, literal \n sequences) missing from the translation, a line break in a name, characters XML cannot store, a translation identical to the source, or the same source text translated differently elsewhere. Blocking problems prevent the export. The work is saved (Ctrl+S) as a sidecar file, fomod/translations/<mod>.<language>.ximod-translation, which the window reloads automatically the next time the folder is opened; Update from folder re-reads the FOMOD and reports new, changed, moved and removed strings, keeping the existing translations (changed sources are marked “to review”).
Export the translated files writes info.xml and ModuleConfig.xml either into a sibling folder fomod_<language> (default, the originals are untouched) or over the original files after a timestamped .bak copy. The export is lossless: only the translated strings change; comments, encoding (UTF-8 or UTF-16), unknown elements and formatting of the original XML are preserved byte for byte. “Keep the original order” forces order="Explicit" so that lists sorted by name are not re-sorted by the translated names. Preview translated opens the installer simulation on the translated copy. The same operations exist on the command line: ximod-architect translate extract | status | update | apply (see § 5).
Translation package — the third export mode builds an archive ready to upload to Nexus: by default a “patch only” package containing just the translated info.xml and ModuleConfig.xml (at their original path and encoding) plus a README in the target language, which users install over the original mod; the “Full mod” option includes every file of the mod instead (check that the author allows redistribution). The format (.zip or .7z) and the file name follow the Name template, with the tokens {name}, {version}, {LANG}, {lang}, {lang3} and {langname} (default “{name}_{LANG}”, e.g. MyMod_FR.zip). The archive is built in the background with the usual progress window.
Productivity — a translation memory per language pair, stored in the configuration folder, learns every translation you save; “Fill from memory” pre-fills identical strings in the next mod (marked “to review”), and the lower panel suggests the remembered translation of the selected string. “Propagate to identical” copies the current translation to every other string with the same source text. The Glossary (toggle in the tools row) lists imposed terms — a term and its expected translation, optionally case-sensitive, or a term to keep untranslated — and any row that breaks a rule is flagged as a problem.
Export CSV / Import CSV exchange the whole table (key, context, source, translation, status, note) with a spreadsheet or another translator; the import matches rows by key and tolerates Excel’s separators.
Unique texts — a toggle in the filter row lists one row per distinct source text, with a ×N badge giving the size of the group; translating that row translates every string with the same text at once, automatically and without asking (locked strings excepted), and a message says how many were updated. It is the quickest way through FOMODs that repeat the same option names or descriptions on many pages. Off, the table shows every string and “Propagate to identical” remains available for a one-off copy to the untranslated ones only.
Keyboard and whitespace — in the table, ↑ and ↓ move the selected row, Page Up and Page Down jump by a screenful of rows, Home and End go to the first and last row (while no translation field is being typed in; the same keys work in the Project strings window). “Show whitespace and line breaks” makes every blank visible in the source text, both in the table and in the detail pane: · for a space, ⍽ for a non-breaking space, → for a tab, ␍ for a carriage return and ¶ at each line end.
On the command line: ximod-architect translate package <root> <sidecar> [-o OUT] [--format zip|7z] [--full], translate export-csv / import-csv, translate tm-learn / tm-apply; and a batch manifest entry may carry a “translations” list (sidecar, mode, out, format, full) to produce the translated packages along with the main archive.

8.14 Opening a mod archive

File → Open archive… (or dropping a .zip/.7z file on the window) extracts the archive into the work folder of the configuration directory (work/<archive name>) in the background, with the usual progress window, then opens the FOMOD found inside — whether the fomod folder sits at the top of the archive or under a single wrapping folder. An archive that was already extracted is reused as is, so the edits you made there are kept. .rar archives are not supported: extract them with 7-Zip first. The translation window has the same button, Open a mod archive…, which extracts only the fomod folder (into

translate_work/<archive name>); the “Full mod” package option is then unavailable, since the mod files are not there.
On the command line: ximod-architect extract <archive> [-o DIR] [--fomod-only] extracts an archive (by default into a folder named after it) and prints the FOMOD root.

8.15 Import fidelity

XIMOD now reads and writes back constructs it used to drop silently: the mod-wide requirements (moduleDependencies, edited in the Mod Info tab under “Mod requirements” with the same dependency editor as elsewhere), the alwaysInstall and installIfUsable attributes of files and folders (the “Always” and “If usable” columns of the file tables), the title position and colour and the header image options showImage, showFade and height (Mod Info → “Advanced header”), and the Ascending/Descending order attributes of steps, groups and options, which are kept as authored.
What still cannot be modelled — elements outside the schema that XIMOD does not know — is reported when the project is loaded: a warning “This FOMOD contains N constructs XIMOD cannot edit; they will be dropped when the project is saved” opens the Problems panel with one line per construct and its location (step, option, conditional set); clicking a line selects the node. The warnings stay in the Problems panel across validations as long as the project is open. ximod-architect validate prints the same warnings.

8.16 Backups and recovery

Each time a project is saved over an existing version, the previous info.xml and ModuleConfig.xml are first copied into fomod/backups/<date-time>/ (nothing is copied when they are identical to what is about to be written), and only the most recent N folders are kept — N being the “Backups to keep” setting. That folder is never included in distribution archives or BA2 packages, and its files are never reported as orphans. Both XML files are written atomically (temporary file, flush, rename), so a crash during a save cannot leave a half-written installer.
File → Restore a backup… lists the backups, newest first, each with the number of differences from the project as it is now; Compare opens the usual comparison window against that backup, Restore loads it into the editor as a single undoable change (nothing is written to disk until you save), and Delete all backups — confirmed by a second click — empties the folder. In addition to the crash guard of § 8.11, a recovery copy of every modified project is now written at a regular interval (“Autosave recovery copy every N minutes”); it is cleared on a normal exit, so the recovery prompt still only appears after an abnormal one.
On the command line: ximod-architect backups <root> --list, --restore <timestamp> or --prune N; build and package also make a backup before writing.

8.17 Masters as automatic conditions

When a plugin file (.esp, .esm or .esl) is added to an option — from the file table or by drag-and-drop — XIMOD reads its header: if the project has no author yet, the plugin’s author is filled in; and every master the plugin requires that is neither one of the game’s own files (listed per game in Categories.json under "masters") nor installed by this mod becomes a file condition “Active” of the option, added to the first dependency pattern of the option’s default type (created if needed). A master already named by a file condition of the option is never added twice. The setting “Add the masters of a plugin as conditions” turns this off.
Validation reads the header of every plugin installed by the project (required files, options, conditional sets) and warns about any master that is neither in the mod, nor a base file of the game, nor declared as a dependency: “X requires Y, which is neither in this mod nor declared as a dependency”. The warning targets the node, so clicking it in the Problems panel selects the option.

8.18 Light plugins (ESL) and the plugin report

Validation also checks the light-plugin rules: a file with the .esl extension whose light flag is not set, a plugin flagged light whose new records exceed the limit or fall outside the allowed FormID range, and — as an information — an .esp that could be flagged light (it counts the records new to the plugin, that is those whose FormID index equals the number of masters, and compares with the game's limit: 2048 by default, 4096 for Starfield, adjustable per game with "eslLimit" in Categories.json). Files above 64 MB are not counted. The CLI inspect <plugin> prints the same figures.
Tools → Plugin report… gathers it all in one table: file, kind, light flag, masters (missing ones in the warning colour), new records against the limit, and eligibility; unreadable files are shown as such.

8.19 Richer simulator and scenarios

The summary page of the installer preview (Project → Preview) now ends with a Final file tree: every file that would land in the game's Data folder, folders expanded from disk with their sizes, and the total install size. When two entries land on the same final path, the higher priority wins (ties go to the later declaration) and the loser is shown struck through, with the winning option on hover. Trees above 20 000 entries are truncated and say so.
The file tree follows the same rules as every tree in XIMOD. A right-click on a folder opens the Expand and Collapse submenus (All / Selected / from Selected, the last one acting on the folder and everything under it, including the contents of installed archives). While no text field has the focus, ↑ and ↓ move a cursor ring from row to row, Home and End jump to the ends, → opens a folder (or steps into an open one), ← closes it (or climbs back to it from a file), and Enter selects in the main window the component that installs the row: the option, the required files or the conditional set — for a folder, the component of its first file.
The preview also opens, as a mod manager does, on a Mod information page before the first step: the header image (honouring the showImage and height options), the title in its colour, the author, the version, the website link and the description from info.xml. Next leads to the first visible step, Back returns to it, and the progress bar counts it as the first page.
A Scenario row lets you save the current selections and file assumptions under a name (stored in fomod/scenarios/<name>.json, by option names so they survive reordering), reload or delete them; a scenario whose selections no longer match any option reports how many were not found. The scenario folder is never included in distribution archives.
Validation now also reports, as information, the steps, options and conditional sets that can never be reached because they test a flag value that no option sets (steps: no option in an earlier step).
On the command line: ximod-architect simulate <root> [--scenario <name|file.json>] [--json] prints the visible steps, the selections, the final install list with sizes and overwrites and the total; a batch manifest entry may carry "scenarios": [...] to replay them (an entry fails when a scenario no longer matches).

8.20 Install size per option

After a validation (which measures the files in the background), each option shows its install size and file count above its file table — missing sources are counted in the warning colour — and the project tree carries the size after the option's name. Sizes are cached by file modification time and refreshed on every validation. ximod-architect validate <root> --sizes prints the totals for the options, the required files and the conditional sets.

8.21 Nexus description generator

Tools → Nexus description… builds the text of the mod's Nexus page from the project: title and description, Requirements (the masters of the installed plugins that the game and the mod do not provide, plus the declared file dependencies), Installation options (one bullet per step, groups with their selection type, options with the first sentence of their description), an installation note, and a Changelog computed against a previous version of the FOMOD chosen with Previous version… (steps, options and files added or removed, metadata changes). Choose BBCode (Nexus) or Markdown, tick the

sections you want, pick the language among the translation sidecars found in the project, then Copy or Save as…. On the command line: ximod-architect nexus-desc <root> [--format bbcode|md] [--previous <root>] [--lang <code>] [-o FILE].

8.22 An editable condition editor

The condition editor of § 8.7 now acts on the project. Every “set by” and “used by” line is a link that selects the node in the main window. Each flag has Rename… (an inline field; the new name is applied to every option that sets the flag and every step visibility, option pattern, conditional set and mod requirement that tests it, as a single undoable change; a name already used by another flag is refused) and Delete all uses. Under each flag, the values it is set to and the values it is tested with are listed: a value tested but never set is shown in the warning colour, a value set but never tested in grey. Validation reports the same findings — a warning for a tested value no option sets (targeting the node that tests it) and an information for a flag that is set but never tested.
The Condition builder at the bottom of the window edits the condition of the node selected in the main window — a step’s visibility, an option’s dependency pattern (choose the pattern, create one, change its type), a conditional file set or the mod requirements. It shows the condition as a sentence (IF flag A = B AND file C is Active THEN …) above one row per condition: kind (flag or file), name and value with suggestions taken from the project (known flags and their values, file names, Active/Inactive/Missing), a remove button and Add condition. Changes are applied live and undoable.
The editor is now an independent window: it can be moved beside the main window or onto a second screen, keeps its position and size from one session to the next, and closes with its title-bar button. The flag list works like the project tree. A right-click on a flag heading opens the Expand (All / Selected) and Collapse (All / Selected) submenus. While no text field has the focus, ↑ and ↓ move a cursor ring from one row to the next — flag headings and their “set by” / “used by” lines —, → unfolds a closed flag (or enters an open one), ← folds an open flag (or climbs back to it from one of its lines), and Enter shows the row in the inspector: the node of a line, or the first site of a flag. In the project tree, Enter does the same: it selects the row under the cursor and the inspector follows.

8.23 Project strings table

Tools → Project strings… lists every name and description of the project (mod information, steps, groups, options) in one table: location, field and text, the text being editable in place (single line for names, several lines for descriptions). A search field, a kind filter (all, names, descriptions) and a Duplicates only toggle narrow the rows; identical texts (ignoring case and spacing) are badged ×N with the other locations on hover, which helps spot copy-pasted descriptions. The find/replace row (match case, whole word) replaces in the current row or in every visible row. Double-clicking a location selects the node in the main window. On the command line: ximod-architect strings <root> [--duplicates] [--csv FILE].

8.24 Reading BSA and BA2 archives

XIMOD now reads the tables of contents of Bethesda archives without unpacking them: BSA versions 103 (Oblivion), 104 (Skyrim, Fallout 3/New Vegas) and 105 (Skyrim Special Edition), and BA2 general (GNRL) and texture (DX10) archives of Fallout 4 and Starfield (versions 1, 2, 3, 7 and 8). Morrowind archives are refused with a clear message. Only the entry names and sizes are read; compressed entries are listed with their packed size.
This feeds three places. Destination-conflict detection (§ 8.1) now looks inside the archives an option installs: two options whose archives contain the same asset are reported as a warning (“the game’s archive load order decides”), and an asset that is both packed and installed loose as an information (“the loose file wins”). The final file tree of the simulator (§ 8.19) lists the contents of each installed archive under it, marked “(in archive)”, and the total shows how much is packed in archives (without counting it twice). And Tools → Archive contents… (or the eye button next to an archive in a file table) opens a table of the entries — format, count, total unpacked size, path, size, compressed — with a

search field. On the command line: ximod-architect archive <file.bsa|file.ba2> [--json]; validate now also prints the certain destination conflicts.
The experimental BA2 writer (CLI ba2) is now validated by reading its output back with the new reader: paths, sizes and name/folder hashes match.

8.25 Nested condition groups and version conditions

Conditions are no longer limited to one flat list with a single operator. Every place that holds a condition — a step's visibility, an option's dependency patterns, a conditional file set and the mod requirements — is now a group that can contain conditions and nested groups, each with its own And / Or operator, to any depth, exactly as the FOMOD schema allows: "(DataVersion = Standard OR DataVersion = Undelayed) AND DataPreset = Aurelia" is kept as written, loaded, edited and saved without change (verified on Aurelia UI and its 234 conditional sets). Two condition kinds were added as well: Game version (gameDependency) and Mod manager version (fommDependency), each with a minimum version.
In the dependency editors, Add group creates a nested group as an indented frame with its own operator, its own conditions, Add condition / Add group / Remove group buttons; the condition builder (§ 8.22) shows nested groups between parentheses in its sentence and edits them with the same controls. The simulator, the validation (flags tested but never set, unreachable steps), the conflict detection and the Nexus description all understand nested groups. Older project templates saved with the flat layout still load.
Two smaller changes in the same release: a menu opened by sliding the pointer along the menu bar now closes when the pointer leaves it (a menu opened by a click stays until something is clicked); and the fonts needed to display the scripts of the project itself — Japanese, Korean, Chinese, Arabic, Thai… in names and descriptions — and of the translation being edited are loaded automatically, whatever the interface language, so no more squares in place of those characters, including in the language lists of the translation and Nexus windows.
The menus are organised into File, Edit, Project, Tools and Help, with a toolbar for the everyday actions, and F1 opens this manual. Group selection types, option types, operators and themes are shown with plain-language labels ("Exactly one (required)", "Recommended", "All conditions (AND)"…) with an explanation on hover, and each section of the option panel has an ⓘ marker explaining the concept. Confirmation dialogs dim the window, accept Enter and Esc, and scroll when the list of issues is long.

8.26 Editing the games and their categories

Tools → Games / categories… opens an independent window to maintain the list of games offered in the Mod Info tab without editing Categories.json by hand. The left pane lists the games; Add game, Delete game and the Move up / Move down buttons change the list and its order. For the selected game the editor shows its identifier (a unique internal key), its name, its Nexus slug (used for the links and the Nexus description), the light-plugin record limit, the base master files (one per line — the game's own masters and official DLC) and the Nexus categories (one per line, in display order). The entries are checked as you type: a game with no identifier, a duplicate identifier or a missing name is flagged, and Save stays disabled until every problem is fixed. Save writes the file — to the shipped assets/data/Categories.json when it is writable, otherwise to a per-user copy that the application reads first — and the new list takes effect immediately; Close discards unsaved edits after a confirmation.

8.27 Tree colours

Tools → Tree colours… gives each kind of row of the project tree its own colour — mod information, steps, groups, options, required files and conditional installs — so the structure of a large installer can be read at a glance, which also helps when eyesight is not what it used to be (combine it with the interface font size, § 4.5). The window lists the six kinds with a swatch, the icon in that colour and the RGB value; Reset colours restores the defaults. There is one set of colours per theme: the window edits the set of the theme currently in use and says so, and switching between dark and light (§ 4.5) switches

the set. The colours are saved in Config.ini (Appendix A). The selected row always keeps the standard text colour so it stays legible on the selection background.
Clicking a swatch opens the Select a colour window, the same picker as in XIMOD Translator. Its Index colour tab shows the AutoCAD colour index: the 24 × 10 grid of hues and shades, the nine standard colours, the grey ramp, and a Colour field to type an index from 1 to 255. Its True colours tab offers a hue / saturation area, a luminance slider, the H, S and L fields, the red, green and blue values of the current colour, and an RGB colour field where a value such as 255,128,0 can be typed. OK applies the colour at once — the tree repaints behind the window — and saves it; Cancel or Esc keeps the previous one.

8.28 File dialogs remember where you were

When you add sources to an option (Add file, Add folder) or browse for an image, the dialog no longer starts from the root of the project every time: it opens in the parent of the last file or folder you chose. Pick Feature D\A for one option and the next dialog opens in Feature D, with B and C right there; pick files inside a folder and the next dialog opens in that folder. The memory lasts for the session and is reset when the project changes; if the remembered folder is outside the root or has disappeared, the dialog falls back to the root. For a large installer the quickest route remains the Explorer: open the feature folder once and drop each subfolder onto its option (§ 8.5).

8.29 Preset wizard

Project → Preset wizard… builds, on the current project, the "preset + customise" pattern explained in § 7.5: a user can pick a preset and be done, or pick a preset and still adjust a few features, or build everything by hand — without any file ever being installed twice. Doing this by hand means flags on every option, dynamic types, visibility conditions and one conditional install per option; the wizard does all of it in one undoable step.
In the From existing pages tab, tick the steps that are feature pages (their options are what the presets choose from), check the generated names — installation-mode step, preset group, "Use the preset" and Customise options, preset flag, names of the "None" and "No preset" options — and define the presets: Add preset creates a column, and the matrix shows every option of every ticked page as a row. In an exclusive group (SelectExactlyOne / SelectAtMostOne) choose one option or (none) per preset; in a multiple-choice group tick any number. The bar at the bottom reports the first problem (no page ticked, no preset, a missing name, two presets with the same name) and Apply stays disabled until it is fixed. Apply inserts the installation-mode step (first, or where an earlier run had left it): the Preset group, a single choice between the presets and No preset (build my own), each writing only the preset flag; then one two-option group per feature page — Use the preset's choices or Customise — which alone writes that page's Customise flag. Every feature page becomes visible only when its Customise option is chosen; the wizard adds the flags to its options, marks the variants a preset chose as Recommended under that preset, adds a "None" option to exclusive groups (recommended by default, optional under the presets that pick something; a SelectAtMostOne group becomes SelectExactlyOne, so that mod managers do not show a second None), and moves the files of the options to the Conditional Installs tab: one set per option, installed when the page is customised and the option ticked there, or when the page is left to the preset and that preset chose the option. No flag is ever written by two options that can be selected together, so going back in the installer and changing anything always installs what the user finally sees (§ 7.5). The message summarises what was done; Edit → Undo reverts the whole operation. Then check the result with Project → Preview installer… (preset alone, preset plus a customised page, page alone) and Project → Validate FOMOD, which knows this pattern (§ 8.1).
The New skeleton tab is for a project that has no pages yet: describe each feature by a page name, the kind of choice (exclusive or multiple) and one variant per line, then Create pages adds them to the project — without files — and switches to the first tab with those pages ticked. Define the presets there, apply, and assign the files afterwards: on the pages for what is chosen by hand, and in the conditional installs created by the wizard for what the presets install.

Running the wizard again on a project it has already transformed does not stack a second structure on the first: the window opens with the existing pages, names and presets loaded and says so. Add or change presets, tick more pages if needed, then Apply rebuilds the whole structure in one undoable step and keeps the flag names, so an update of the mod stays compatible with its previous release. A structure made by version 2.1.0 or 2.2.1 is recognised too and rebuilt the current way: run the wizard again, apply, and publish the update.

8.30 Cut, copy and paste between FOMODs

Steps, groups and options can be cut, copied and pasted: Edit → Cut, Copy and Paste, the same commands in the right-click menu of a node of the tree, or Ctrl+X, Ctrl+C and Ctrl+V while no text field has the focus — in a text field these keys keep acting on the text. A copy takes the node with everything it contains: groups and options, files, images, flags, conditions and dependency patterns. Cut removes the node from the project and keeps it in the clipboard; Edit → Undo puts it back.
Paste follows the selection: a step is inserted after the selected step (at the end when nothing is selected), a group goes into the selected step, after the selected group if there is one, and an option into the selected group, after the selected option. The pasted node is selected, and the paste is one undoable change. When the selection cannot receive the node — a group with no step selected, an option with no group — a message says what to select.
The clipboard belongs to the application, not to a project: copy in one FOMOD tab, switch to another tab and paste. A copy is also placed on the system clipboard, so a node copied in one XIMOD window can be pasted in another. Paths are relative to the mod folder, so a node brought from another mod may point at files this mod does not have: after the paste, XIMOD reports how many of its files are missing from this mod folder (Project → Find missing files…, § 8.32, then helps to re-point them) and lists the flags the node tests that no option of this project sets — a page that would never appear, or an option whose type would never change, is caught at once.

8.31 Renaming a source on disk

Renaming a file or a folder in the file explorer breaks every reference the project holds to it. The Rename on disk… button of the file tables — the Source table of an option (§ 4.2), the Required Installs tab (§ 4.3) and the files of a conditional set (§ 4.4) — renames the selected file or folder from XIMOD instead, and updates the references at the same time: every option, required file and conditional set that uses it and, for a folder, every reference to something inside it. The button is available once the project has a root directory.
The window shows the current path, a field for the new name (the item stays in the same folder) and the number of references that will follow; Rename or Enter applies. A name containing a character that Windows forbids (such as : * ? or |) or ending with a dot or a space, a name already taken in that folder, or a source that no longer exists is reported in the window, which stays open. A change of capitalisation only (main.esp → Main.esp) works on every system, and each reference keeps its separator style (\ or /). The item is renamed on disk at once: Edit → Undo restores the references of the project, not the name on disk.

8.32 Finding missing files

Files are also renamed and moved outside XIMOD — while reorganising a mod, or after unpacking a new version of it. Project → Find missing files… scans the mod folder and lists every file, folder and image the project uses that no longer exists, once per path, with the number of references that use it. For each one, the window proposes candidates found in the mod folder: the same name in another folder (moved), or, in the folder where it was, a similar name that the project does not use yet, with the same extension for a file (renamed). When exactly one item of the same name exists elsewhere, it is pre-selected. Choose another candidate in the list, keep (leave as is), or click Browse… to pick any file or folder of the mod folder.
Apply updates every chosen reference at once — for a folder, the references inside it too — as one undoable change, then scans the mod folder again; Rescan does the same after you have moved files

yourself. Unlike a file watcher, this also catches what changed while XIMOD was closed, and it never alters the project without your confirmation. Project → Validate FOMOD (§ 4.8) still reports the files that remain missing.

Appendices

Appendix A — The Config.ini file

The configuration is saved in the Config.ini file, located next to the executable. Here is a complete example:

```
; XIMOD Architect Configuration
[General]
Locale=fra
Country=FRA
FirstStart=1
Theme=Dark
FontSize=14
ReplaceNewlines=1
MaxRecentFiles=10
SplashScreenSeconds=2

[Window]
WindowWidth=1280
WindowHeight=800

[WindowPositions]
WinPos_ximod_translation=120,80
WinSize_ximod_translation=1100,700
WinPos_ximod_preview=200,140
WinSize_ximod_preview=900,650

[Scripts]
PreSaveScript=
PostSaveScript=

[RecentFiles]
RecentFile0=C:\Mods\MyMod
```

Two keys are added to the previous ones: `Country` (ISO 3166-1 alpha-3 code of the chosen country) and `FirstStart` (see § 2.5).

The `[WindowPositions]` section remembers the geometry of the free windows (see § 2.6). For each window, `WinPos_<id>` stores its position (x,y) and `WinSize_<id>` its size (width,height). These entries are only created once the user has actually moved or resized the window; without them, the window opens centred on the main window. The size of the main window is kept in `[Window]` (WindowWidth / WindowHeight), but its position is not.

Appendix B — Keyboard shortcuts

On macOS, the Ctrl key corresponds to Cmd.

Global shortcuts

Shortcut	Action
Ctrl+N	New project.
Ctrl+O	Open a folder.
Ctrl+Shift+O	Open a file.

Shortcut	Action
Ctrl+S	Save (generate the FOMOD).
Ctrl+,	Open the Settings.
Ctrl+Q	Quit the application.
F1	Open the user manual (Help → User manual).
Ctrl+X / Ctrl+C / Ctrl+V	Cuts, copies or pastes the step, group or option selected in the tree (§ 8.30). In a text field these keys act on the text.

Windows and dialogs

Shortcut	Action
Esc	Closes the active free window (Country selection, Properties, XML editor) or the modal dialog (Settings, About, confirmation, validation report, scripts). On a confirmation, acts as "Cancel". No effect on the translation editor or the main window.

List navigation — Properties and translation editor

Key	Action
↑ / ↓	Moves the selection one row up / down.
Page Up / Page Down	Scrolls one screen; the step adjusts to the window height.
Home / End	Jumps to the first / last row (the last row is brought fully into view).
Enter	Properties: selects the highlighted item. Translation editor: places the cursor in the row's input field.
Tab / Shift+Tab	Properties: next / previous row (never on the flag). Translation editor (while editing): next / previous translation field.

Flag grid — "Country selection" window

Key	Action
↑ ↓ ← → (or 8 2 4 6)	Moves the selection frame within the grid.
Page Up / Page Down	Scrolls one screen of flags.
Home / End	Jumps to the first / last flag (the last row is brought fully into view).
Tab / Shift+Tab	Next / previous flag.
Enter	Confirms the highlighted country.
Esc	Closes the window.

Appendix C — Glossary

Term	Definition
FOMOD	Mod installer format based on info.xml and ModuleConfig.xml; the standard for Bethesda games, and supported for most other games by Vortex and Mod Organizer 2.
Step	Page of the installation wizard, containing one or more groups.
Group	Set of options (plugins) sharing a selection type.
Plugin	Option offered to the user, associated with files to install.

Term	Definition
Flag	Condition flag (name = value pair) used to drive the installation.
Endonym	Name of a country or language as it is written in that language.
ISO 639-3	Three-letter language code naming the translation folders (e.g. fra, eng).
ISO 3166-1 alpha-3	Three-letter country code (e.g. FRA, USA).
Root directory	Working folder containing the mod files and the generated fomod subfolder.

Appendix D — Complete FOMOD example

This appendix presents a fictional but complete project — "Dawnlight", a lighting mod for Skyrim Special Edition — that puts into practice the full range of XIMOD Architect's possibilities: required files, multiple steps, a step visibility condition, the five group types, the five option types, condition flags, dynamic option type (dependency patterns) and conditional installs. The two files below form a single project and are reproduced as XIMOD generates them (simplified indentation).

D.1 info.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<fomod>
    <Name>Dawnlight</Name>
    <Author>Patrice</Author>
    <Version>1.2.0</Version>
    <Website>https://www.nexusmods.com/skyrimspecialedition/mods/98765</Website>
    <Description>Improved lighting, candles and weather for Skyrim SE.</Description>
    <Groups>Visuals and Graphics</Groups>
    <Game>skyrimSpecialEdition</Game>
</fomod>
```

D.2 ModuleConfig.xml

Features covered by the example:

- **Required files**: `requiredInstallFiles` (a file and a folder).
- **Step 1**: `SelectAll` group (option `Required`) and `SelectExactlyOne` (options `Recommended` / `Optional`, res flag).
- **Step 2**: `SelectAny` group (candles and moon flags) and `SelectAtMostOne`.
- **Step 3**: visibility condition (`<visible>`) then `SelectAtLeastOne` group with a dynamic-type option (dependencyType: `Recommended` if candles=On, `NotUsable` if a file is missing) and a `CouldBeUsable` option.
- **Conditional installs**: two patterns (`And` and `Or` operators, flag and file dependencies, `Active` / `Inactive` / `Missing` states).

```xml
<?xml version="1.0" encoding="utf-8"?>
<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:noNamespaceSchemaLocation="http://qconsulting.ca/fo3/ModConfig5.0.xsd">
  <moduleName>Dawnlight</moduleName>
  <moduleImage path="fomod\header.png"/>
  <requiredInstallFiles>
    <file source="Dawnlight.esp" destination="Dawnlight.esp" priority="0"/>
    <folder source="core\meshes" destination="meshes" priority="0"/>
  </requiredInstallFiles>
  <installSteps order="Explicit">
    <installStep name="Main installation">
      <optionalFileGroups>
        <group name="Core module" type="SelectAll">
          <plugins order="Explicit">
            <plugin name="Base files">
              <description>Scripts and meshes essential to the mod.</description>
              <files>
                <folder source="core\scripts" destination="scripts" priority="0"/>
              </files>
              <typeDescriptor>
                <type name="Required"/>
              </typeDescriptor>
            </plugin>
          </plugins>
        </group>
        <group name="Texture resolution" type="SelectExactlyOne">
          <plugins order="Explicit">
            <plugin name="2K">
```

```
            <description>2K textures, good quality/performance balance.</description>
            <image path="fomod\2k.png"/>
            <conditionFlags>
              <flag name="res">2K</flag>
            </conditionFlags>
            <files>
              <folder source="tex\2k" destination="textures" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Recommended"/>
            </typeDescriptor>
          </plugin>
          <plugin name="4K">
            <description>4K textures, for powerful setups.</description>
            <image path="fomod\4k.png"/>
            <conditionFlags>
              <flag name="res">4K</flag>
            </conditionFlags>
            <files>
              <folder source="tex\4k" destination="textures" priority="1"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
  <installStep name="Ambience options">
    <optionalFileGroups>
      <group name="Lighting effects" type="SelectAny">
        <plugins order="Explicit">
          <plugin name="Dynamic candles">
            <description>Adds candles with flickering light.</description>
            <conditionFlags>
              <flag name="candles">On</flag>
            </conditionFlags>
            <files>
              <folder source="opt\candles" destination="meshes" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
          <plugin name="Enhanced moon">
            <description>Increases the intensity of the moonlight.</description>
            <conditionFlags>
              <flag name="moon">On</flag>
            </conditionFlags>
            <files>
              <folder source="opt\moon" destination="textures" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
      <group name="Sound ambience" type="SelectAtMostOne">
        <plugins order="Explicit">
          <plugin name="Night sounds">
            <description>Adds a nighttime sound ambience.</description>
            <files>
              <folder source="opt\sound" destination="sound" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
  <installStep name="ENB compatibility">
    <visible>
      <dependencies operator="And">
```

```
        <flagDependency flag="res" value="4K"/>
        <fileDependency file="KnownConflict.esp" state="Inactive"/>
      </dependencies>
    </visible>
    <optionalFileGroups>
      <group name="ENB preset" type="SelectAtLeastOne">
        <plugins order="Explicit">
          <plugin name="Light ENB">
            <description>Low-cost ENB preset.</description>
            <files>
              <file source="enb\light.ini" destination="enblocal.ini" priority="0"/>
            </files>
            <typeDescriptor>
              <type name="Optional"/>
            </typeDescriptor>
          </plugin>
          <plugin name="Ultra ENB">
            <description>High-end ENB preset (dynamic type).</description>
            <files>
              <file source="enb\ultra.ini" destination="enblocal.ini" priority="1"/>
            </files>
            <typeDescriptor>
              <dependencyType>
                <defaultType name="Optional"/>
                <patterns>
                  <pattern>
                    <dependencies operator="And">
                      <flagDependency flag="candles" value="On"/>
                    </dependencies>
                    <type name="Recommended"/>
                  </pattern>
                  <pattern>
                    <dependencies operator="Or">
                      <fileDependency file="EnbHelper.dll" state="Missing"/>
                    </dependencies>
                    <type name="NotUsable"/>
                  </pattern>
                </patterns>
              </dependencyType>
            </typeDescriptor>
          </plugin>
          <plugin name="Experimental extension">
            <description>Additional effects, not guaranteed.</description>
            <files>
              <folder source="enb\extra" destination="textures" priority="2"/>
            </files>
            <typeDescriptor>
              <type name="CouldBeUsable"/>
            </typeDescriptor>
          </plugin>
        </plugins>
      </group>
    </optionalFileGroups>
  </installStep>
</installSteps>
<conditionalFileInstalls>
  <patterns>
    <pattern>
      <dependencies operator="And">
        <flagDependency flag="res" value="4K"/>
        <flagDependency flag="candles" value="On"/>
      </dependencies>
      <files>
        <folder source="cond\candles4k" destination="textures" priority="0"/>
      </files>
    </pattern>
    <pattern>
      <dependencies operator="Or">
        <flagDependency flag="moon" value="On"/>
        <fileDependency file="SkyUI_SE.esp" state="Active"/>
      </dependencies>
      <files>
        <file source="cond\patch.esp" destination="patch.esp" priority="0"/>
      </files>
    </pattern>
  </patterns>
</conditionalFileInstalls>
```

```
</config>
```

Appendix E — License and credits

The source code of XIMOD Architect is distributed under the MIT license (© 2025-2026). The third-party components below retain their own license; the full details and attributions appear in the THIRD-PARTY-NOTICES file provided with the application.

Component	Source	License
XIMOD Architect code	—	MIT
Noto fonts	Google Fonts / Noto project	SIL Open Font License 1.1
Flags (SVG)	drapeauxdespays.fr	Royalty-free (commercial use included)
ISO 3166-1 alpha-3 country codes	ISO — iso.org/obp	Free access
ISO 639-3 language codes	SIL — iso639-3.sil.org	Attribution required (see below)
Rust libraries	crates.io ecosystem	MIT / Apache-2.0 (mostly)

ISO 639-3 attribution: the language codes come from iso639-3.sil.org (source to be credited). The data added by XIMOD — endonyms, associated font, country ↔ language mappings — is not part of the ISO 639-3 standard, and the standard's identifiers are not modified. The application must moreover not provide any means of redistributing the full set of codes.